

Lecture 9 - Convolutional Neural Networks (CNNs)

Ertunc Erdil

April 2025

Introduction to neural networks

$$y = f^*(x)$$

Introduction to neural networks

$$y = f^*(x) \approx g(x; \theta)$$

Introduction to neural networks

$$y = f^*(x) \approx g(x; \theta)$$

set of learnable parameters

Introduction to neural networks

$$\theta^* = \operatorname{argmin}_{\theta} L(y, g(x; \theta)) + R(\theta)$$

Introduction to neural networks

$$g(x; \theta) = g_L \circ \cdots \circ g_2 \circ g_1(x)$$

Introduction to neural networks

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

F. ROSENBLATT

Cornell Aeronautical Laboratory

If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

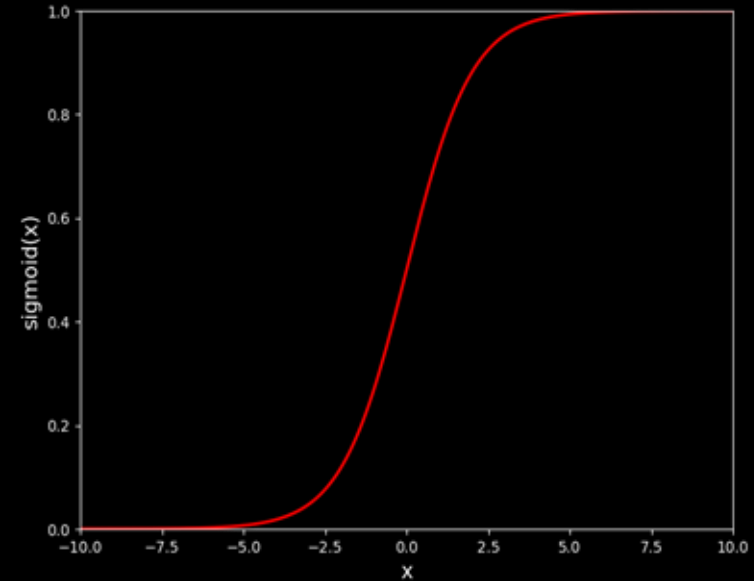
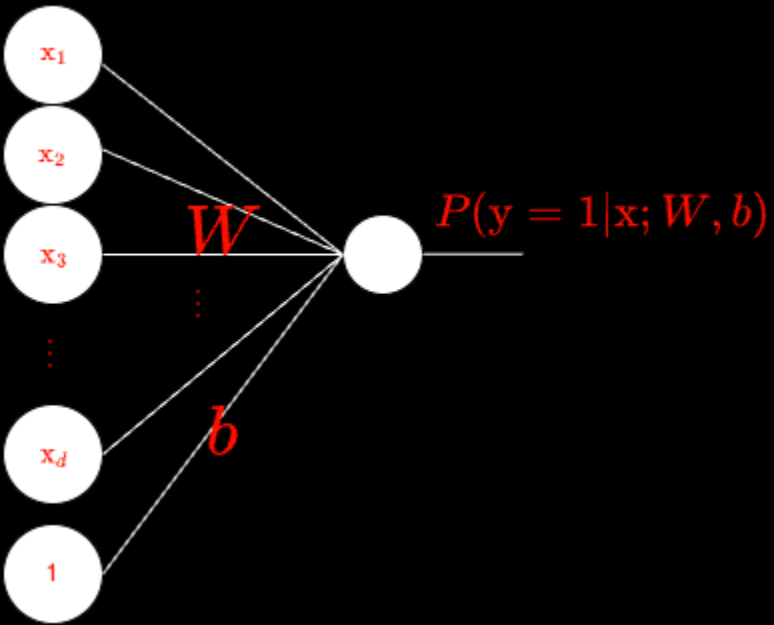
1. How is information about the physical world sensed, or detected, by the biological system?
2. In what form is information stored, or remembered?
3. How does information contained in storage, or in memory, influence recognition and behavior?

and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an organism remembers by reconstructing the original sensory patterns from the "memory traces" which they have left, much as we might develop a photographic negative, or translate the pattern of electrical charges in the "memory" of a digital computer. This hypothesis is appealing in its simplicity and ready intelligibility, and a large family of theoretical brain

Introduction to neural networks

- Perceptron

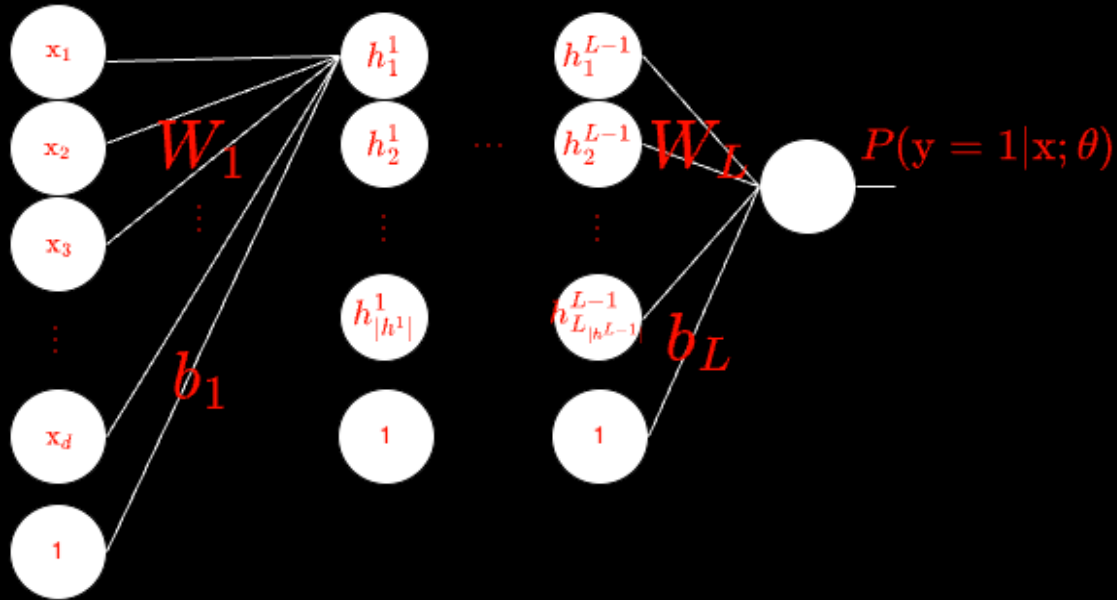
$$g(x; \theta) = g_1(x; W, b) = \sigma(Wx + b)$$



Introduction to neural networks

- Multi-layer Perceptron (MLP)

$$g(x; \theta) = g_L(h^{L-1}; W_L, b_L) \circ \dots \circ g_1(x; W_1, b_1)$$



$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

$$h^l \in \mathcal{R}^{d_l \times 1}$$

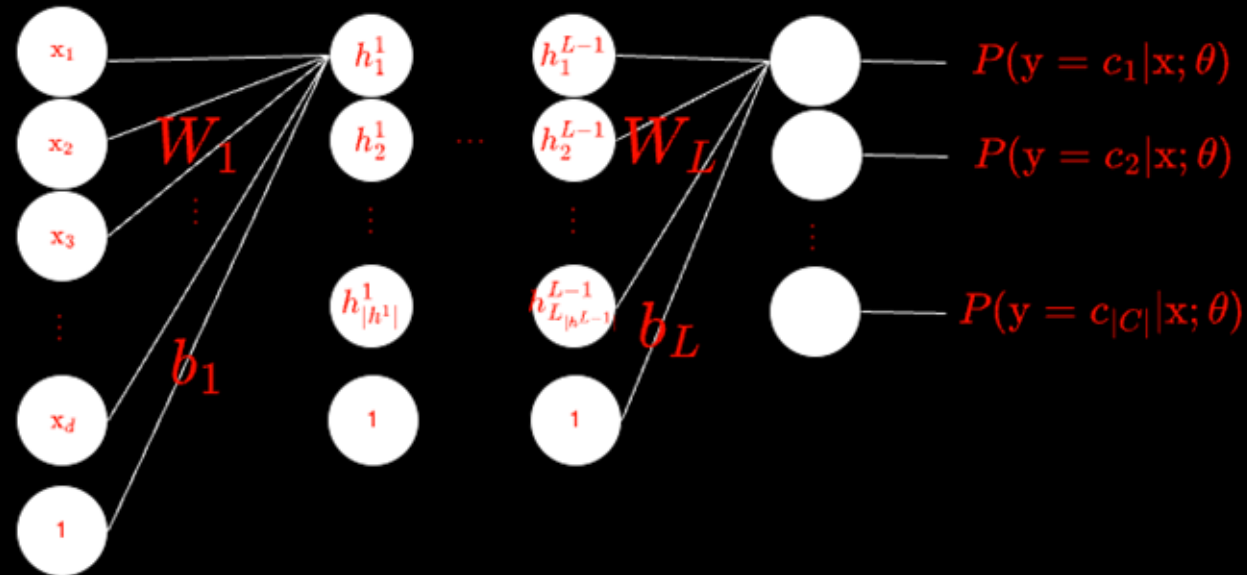
$$h^0 = x \qquad h^L = g(x; \theta)$$

σ_L is Sigmoid

Introduction to neural networks

- Multi-layer Perceptron (MLP) - Multi-class

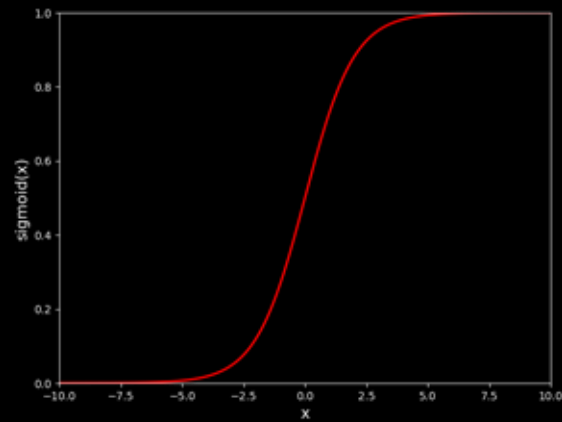
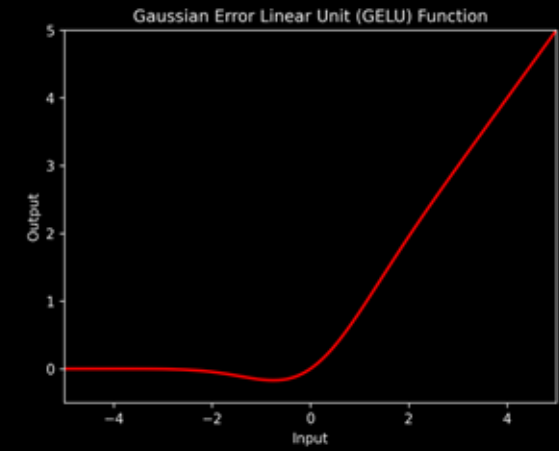
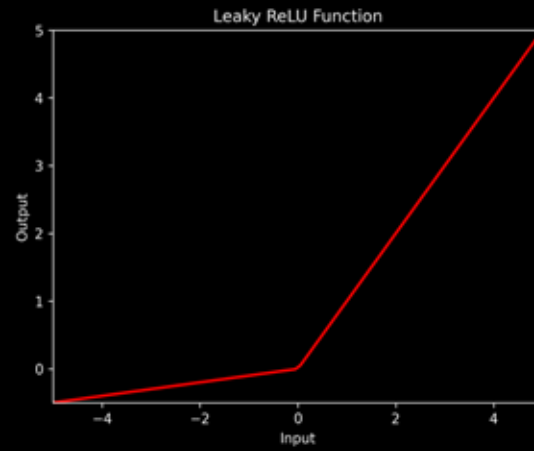
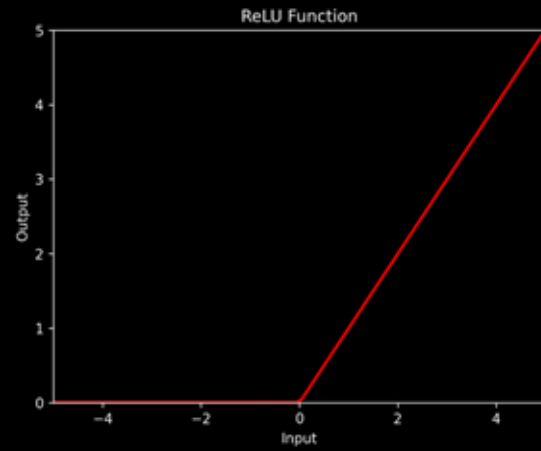
$$g(x; \theta) = g_L(h^{L-1}; W_L, b_L) \circ \dots \circ g_1(x; W_1, b_1)$$



σ_L is Softmax

Introduction to neural networks

- Activation functions σ_L



Introduction to neural networks

- This lecture is about Convolutional Neural Networks (CNNs)
- Recall how we define a single layer in Perceptron or MLP

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

- We define a single layer in CNN as follows

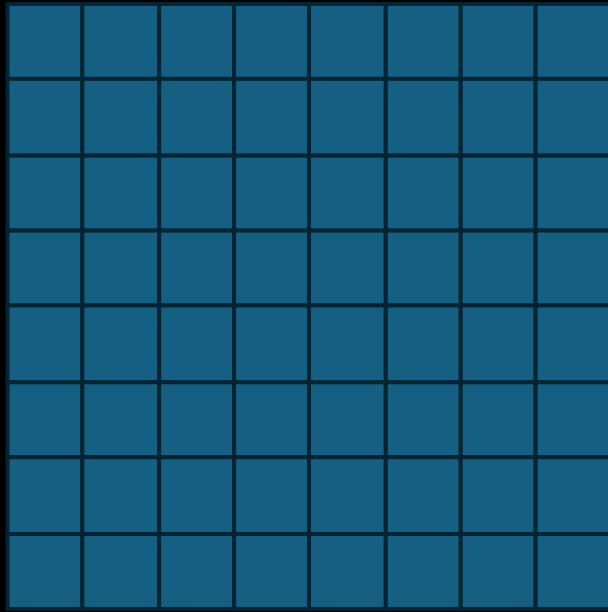
$$h^l = \sigma_l(W_l * h^{l-1} + b_l)$$

Outline of this lecture

- Motivation
- Convolutional layers
- Strides
- Pooling
- A simple image classification CNN

Motivation

- Using MLPs with grid-like structures



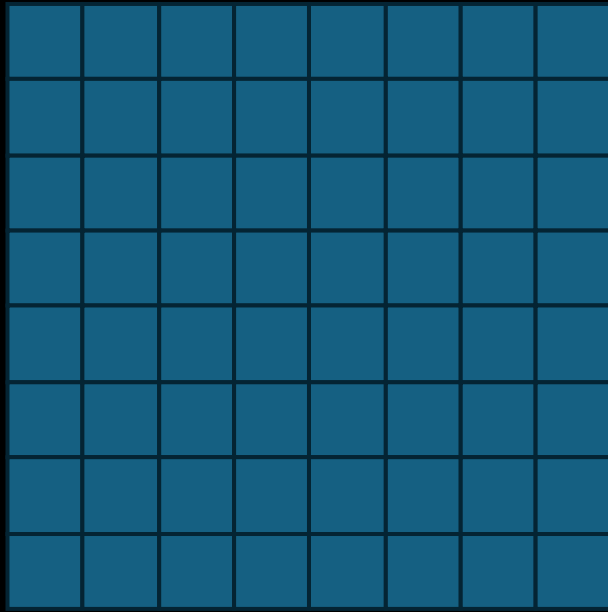
Image



Vectorized image

Motivation

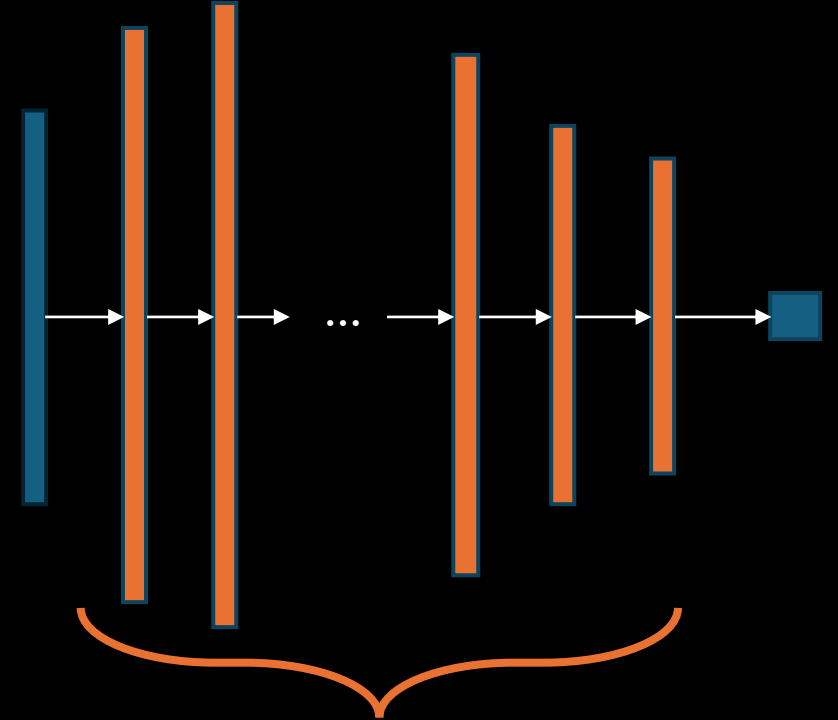
- Using MLPs with grid-like structures



Image



Vectorized image



Hidden layers

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

Motivation

- Problems Using MLPs with grid-like structures

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

- Fully connected links leads to too many parameters

$$x \in \mathcal{R}^{N_0 \times M_0} \quad W_1 \in \mathcal{R}^{(d_1) \times (N_0 \times M_0)}$$

$$(N_0, M_0) = (64, 64) \quad d_1 = 128$$

$64 \times 64 \times 128 = 524,288$ parameters (w/o considering bias term)

With a huge compression from 64×64 dimensions to 128!

Motivation

- Problems Using MLPs with grid-like structures

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

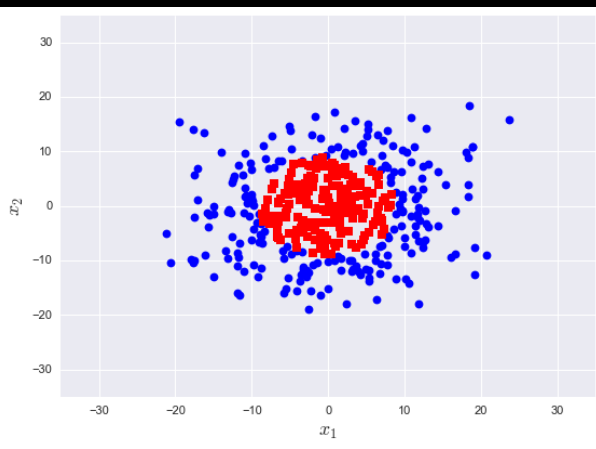
- Fully connected links leads to too many parameters

$64 \times 64 \times 128 = 524,288$ parameters (w/o considering bias term)

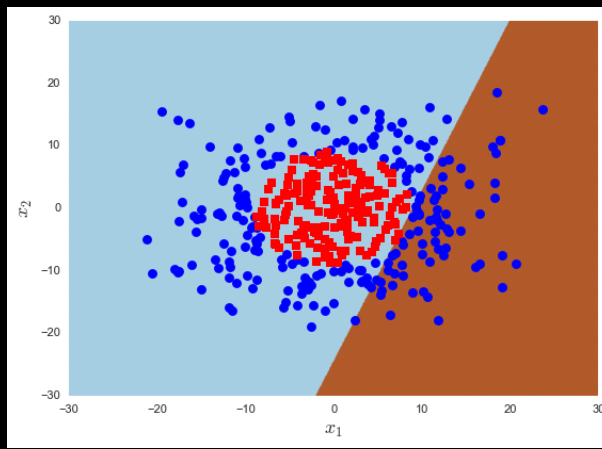
With a huge compression from 64×64 dimensions to 128!

- We need to lift the data to a higher dimension!

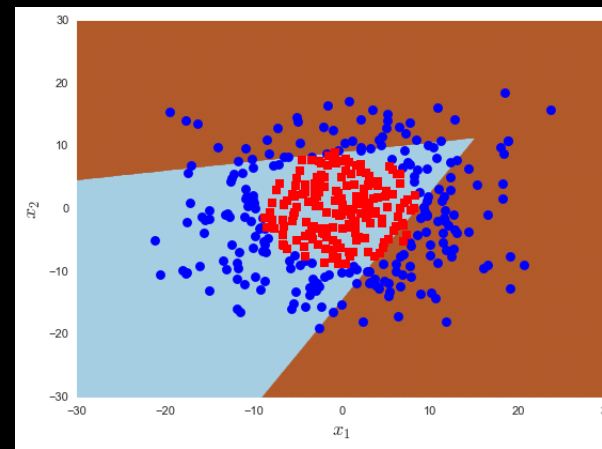
data



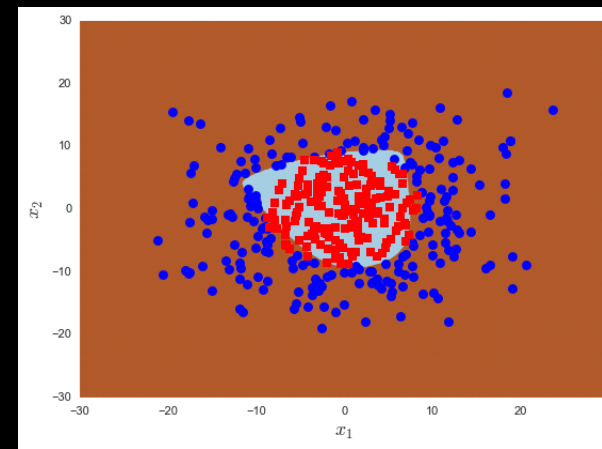
$d_1 = 1$



$d_1 = 2$



$d_1 = 3$

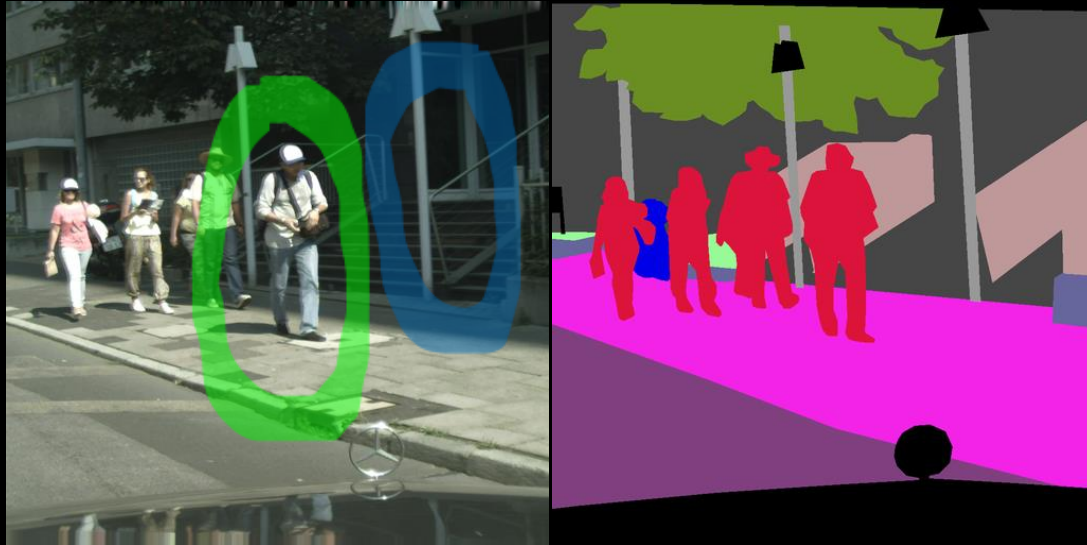


Motivation

- Problems Using MLPs with grid-like structures

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

- Fully connected links leads to too many parameters
- Images are composed of a hierarchy of local statistics

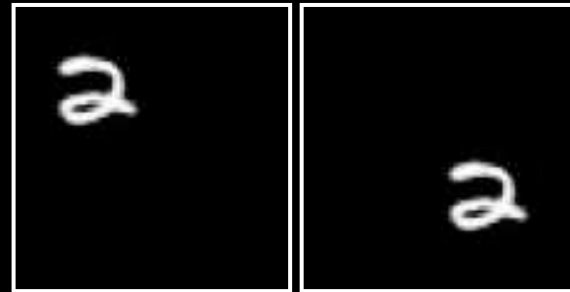


Motivation

- Problems Using MLPs with grid-like structures

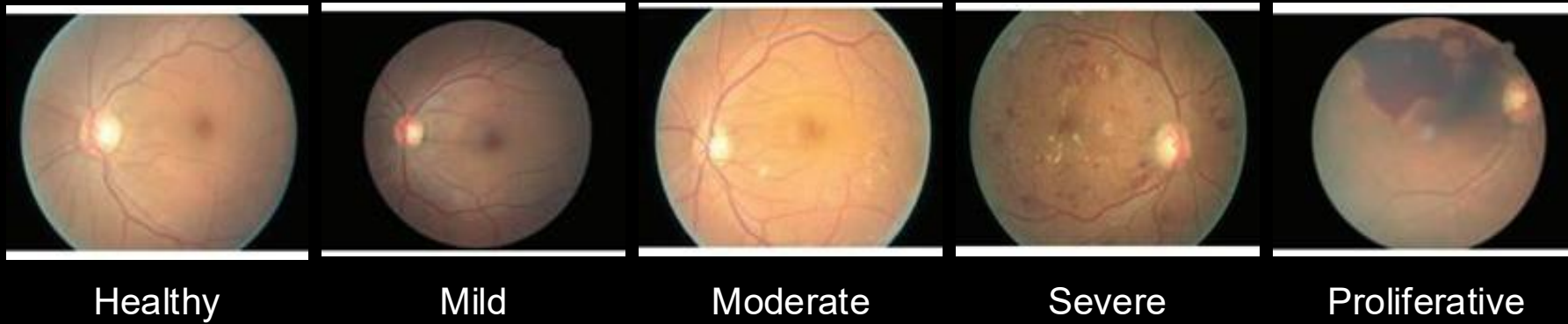
$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

1. Fully connected links leads to too many parameters
2. Images are composed of a hierarchy of local statistics
3. Lack of translation invariance



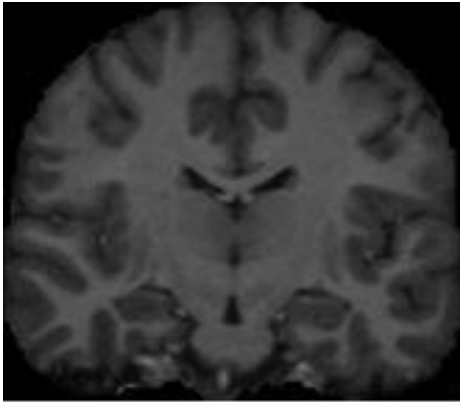
Motivation

- Application of CNNs – Image classification

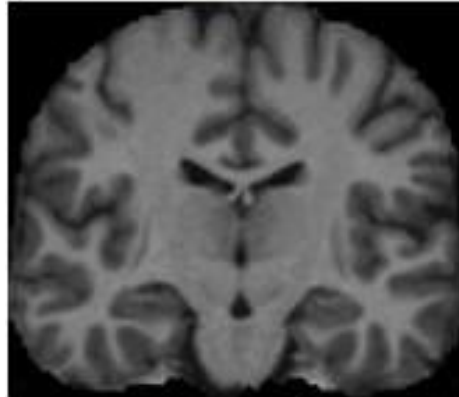


Motivation

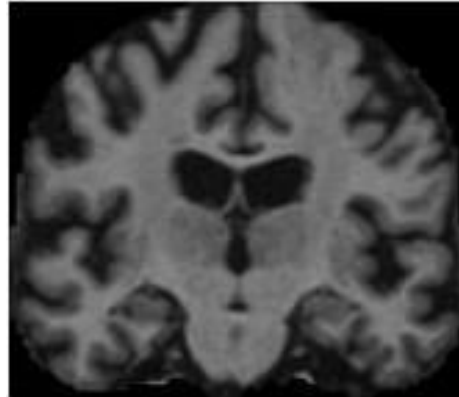
- Application of CNNs – Age regression



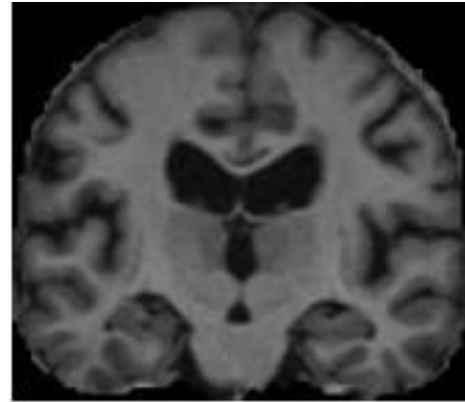
17 years old



30 years old



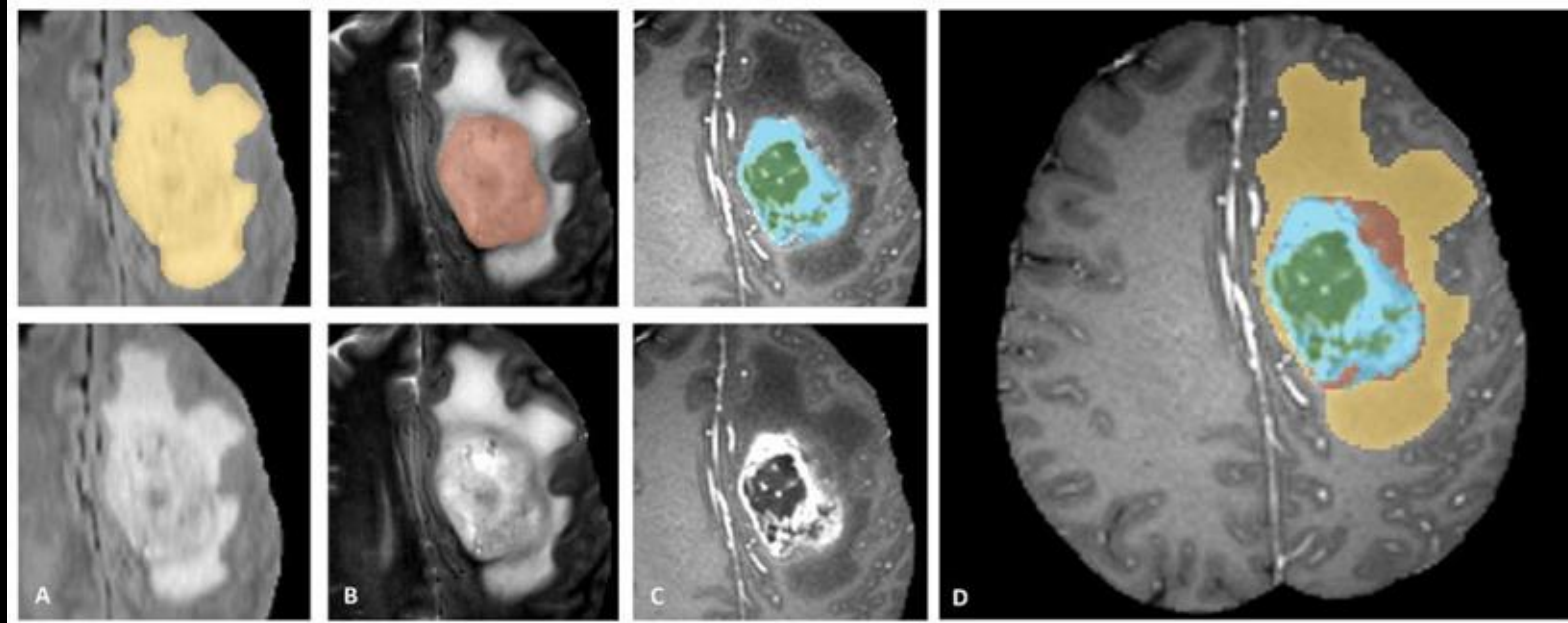
74 years old



81 years old

Motivation

- Application of CNNs – Image segmentation



Outline of this lecture

- ✓ Motivation
- Convolutional layers
- Strides
- Pooling
- A simple image classification CNN

Convolutional layers

- Recall: Convolution is a mathematical operator on two functions
- Convolution operator – 1D
- Consider two functions of a single-variable $f(t)$ and $g(t)$
- Continuous case: $(f * g)(t) = \int f(\tau)g(t - \tau)$
- Discrete case: $(f * g)(t) = \sum_{-\infty}^{+\infty} f(\tau)g(t - \tau)$
- Thus, in words, convolution is the integral of the product of the two functions, after one of them is reversed and shifted

Convolutional layers

- Convolution operator – 2D
- Let's consider the convolution for two function of two variables $I(x, y)$ image, and $k(x, y)$ convolutional kernel
- Discrete case: $(I * k)(x, y) = \sum_m \sum_n I(m, n)k(x - m, y - n)$
- Convolution is commutative (this property is useful in some proofs)

$$(I * k)(x, y) = (k * I)(x, y) = \sum_m \sum_n I(x - m, y - n)k(m, n)$$

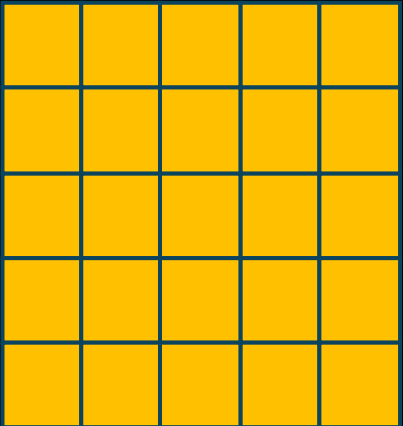
- This commutative property is the only reason why we need to reverse one of the functions before shifting it. In practice, this is not very useful.

Convolutional layers

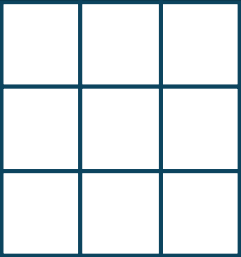
- Convolution vs. Cross-correlation
- Convolution: $(k * I)(x, y) = \sum_m \sum_n I(x - m, y - n)k(m, n)$
- Cross-correlation: $(k * I)(x, y) = \sum_m \sum_n I(x + m, y + n)k(m, n)$
- In practice, in many machine learning libraries, cross-correlation is implemented under the name of convolution.
- In the literature, convolutional kernels are also referred to as convolutional weights and convolutional filters.

Convolutional layers

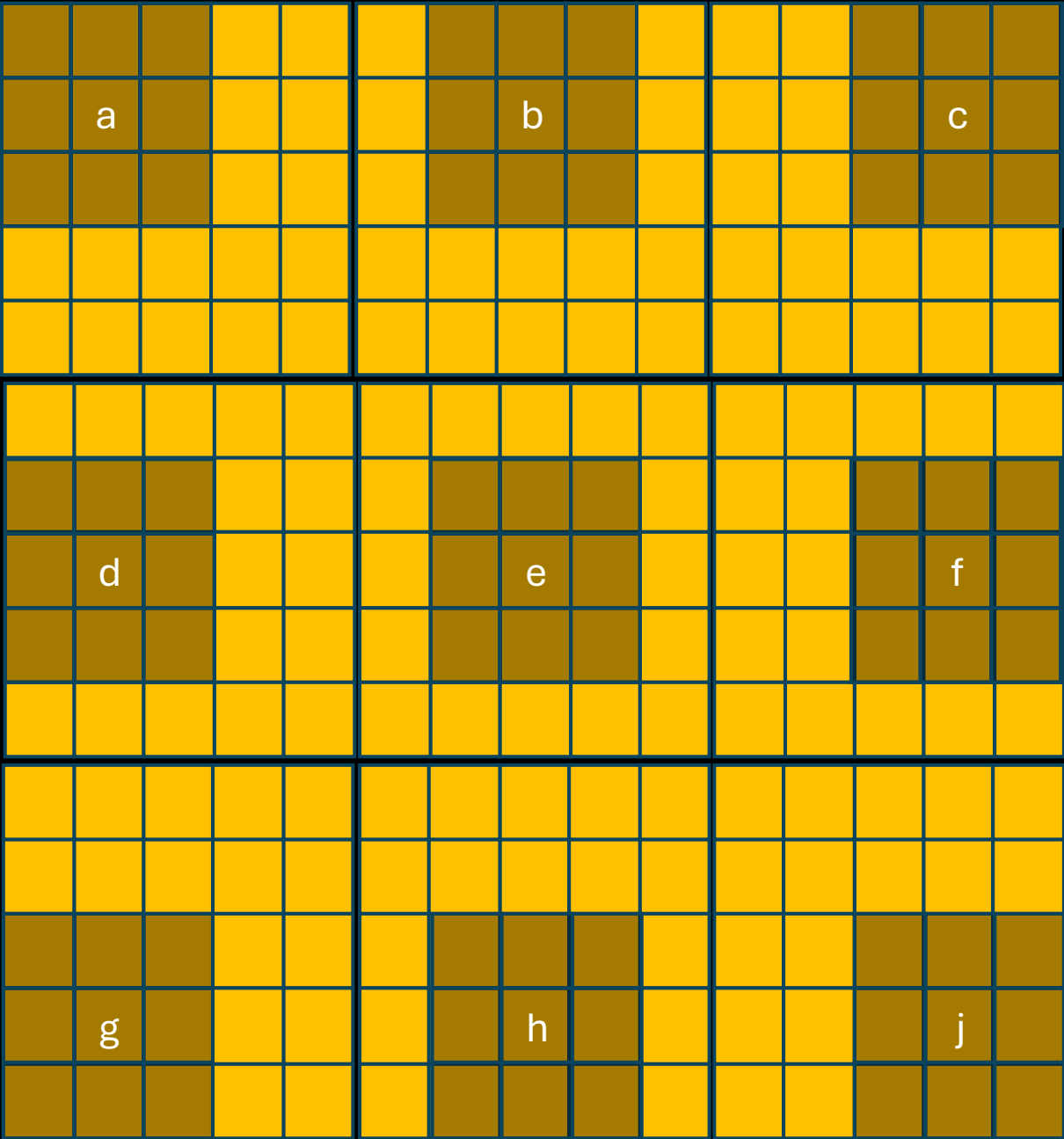
- Visualizing image convolution



Image



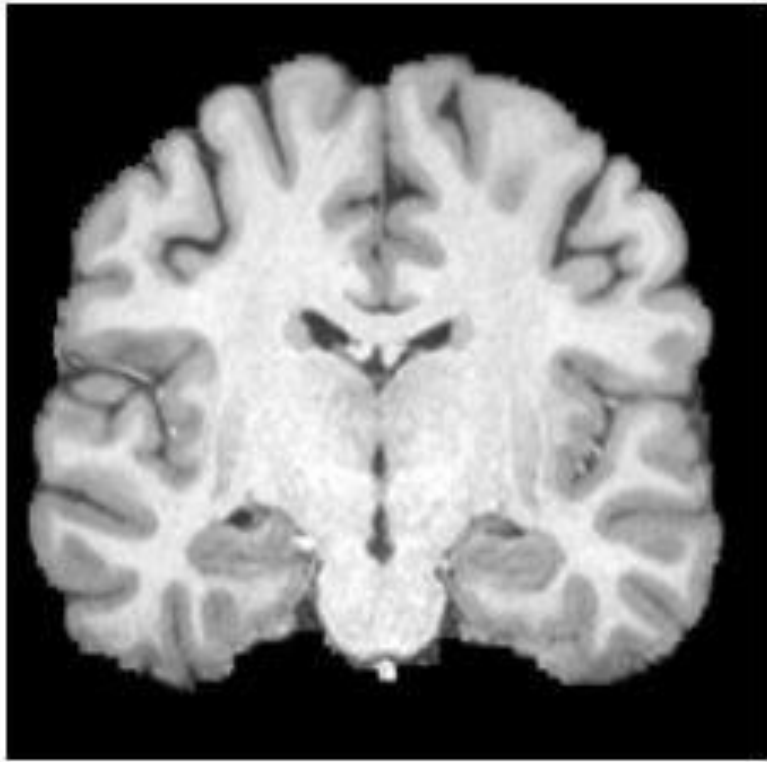
kernel



a	b	c
d	e	f
g	h	j

Convolutional layers

- Convolution example

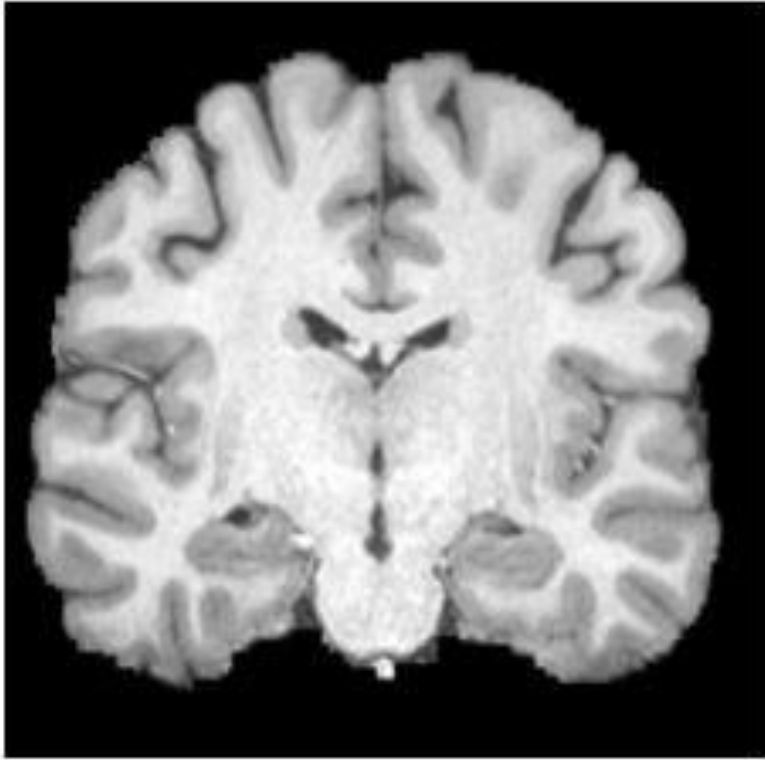


$$\begin{matrix} * & \begin{bmatrix} -3, 0, 3 \\ -9, 0, 9 \\ -3, 0, 3 \end{bmatrix} & = \end{matrix}$$

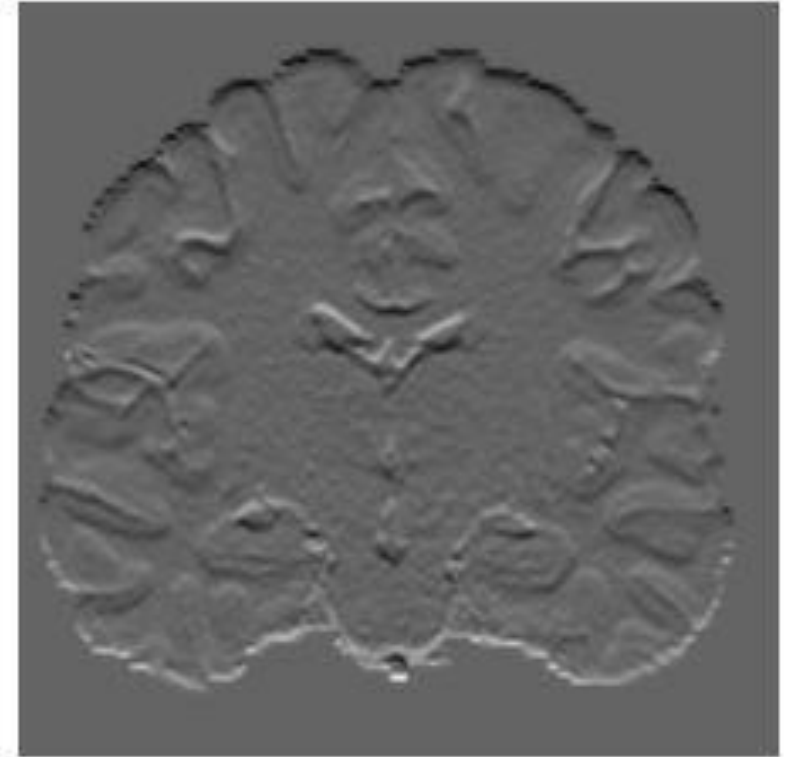


Convolutional layers

- Convolution example

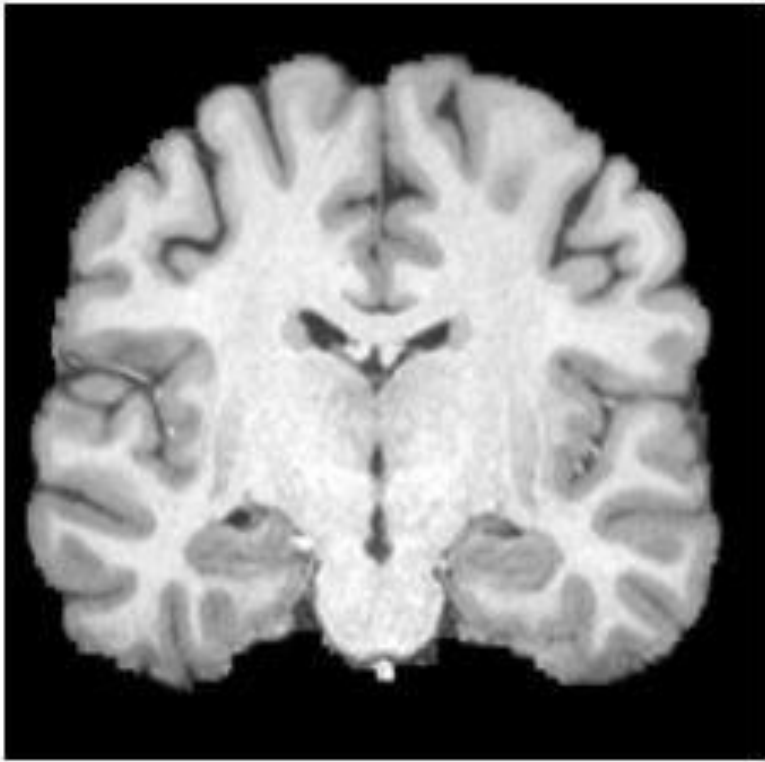


$$\begin{aligned} &[-3, -9, -3 \\ * &+0, +0, +0 = \\ &+3, +9, +3] \end{aligned}$$

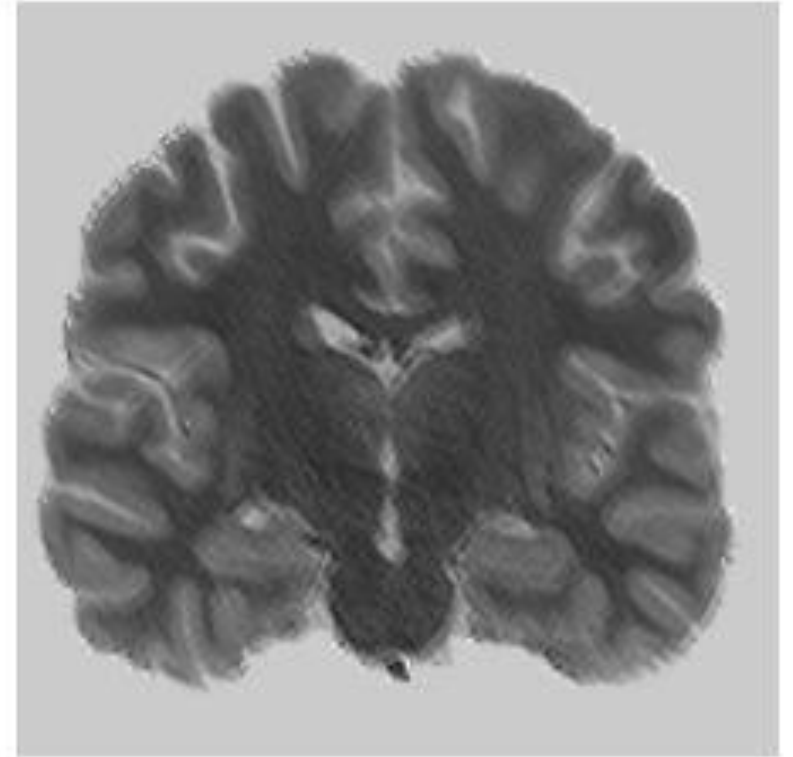


Convolutional layers

- Convolution example



$$\begin{aligned} & [0.2, -0.9, 0.6 \\ & * -0.7, 0.5, 0.1 = \\ & 0.4, 0.0, -0.8] \end{aligned}$$



Convolutional layers

- Convolutions instead of projections

$$a_{l,k} = \sum_j w_{l,kj} h_{l-1,j} + b_{l,k}$$

- Each $h_{l-1,j}$ is a number corresponding to the values of the j^{th} neuron at layer $l - 1$, and so is $a_{l,k}$
- Each h_l is a vector of neurons and a_l is a vector of activation before applying the activation function
- There is a separate $w_{l,kj}$ that is linking each neuron $h_{l-1,j}$ to each $a_{l,k}$
- High dimensions of h_{l-1} and a_l lead to high number of weights

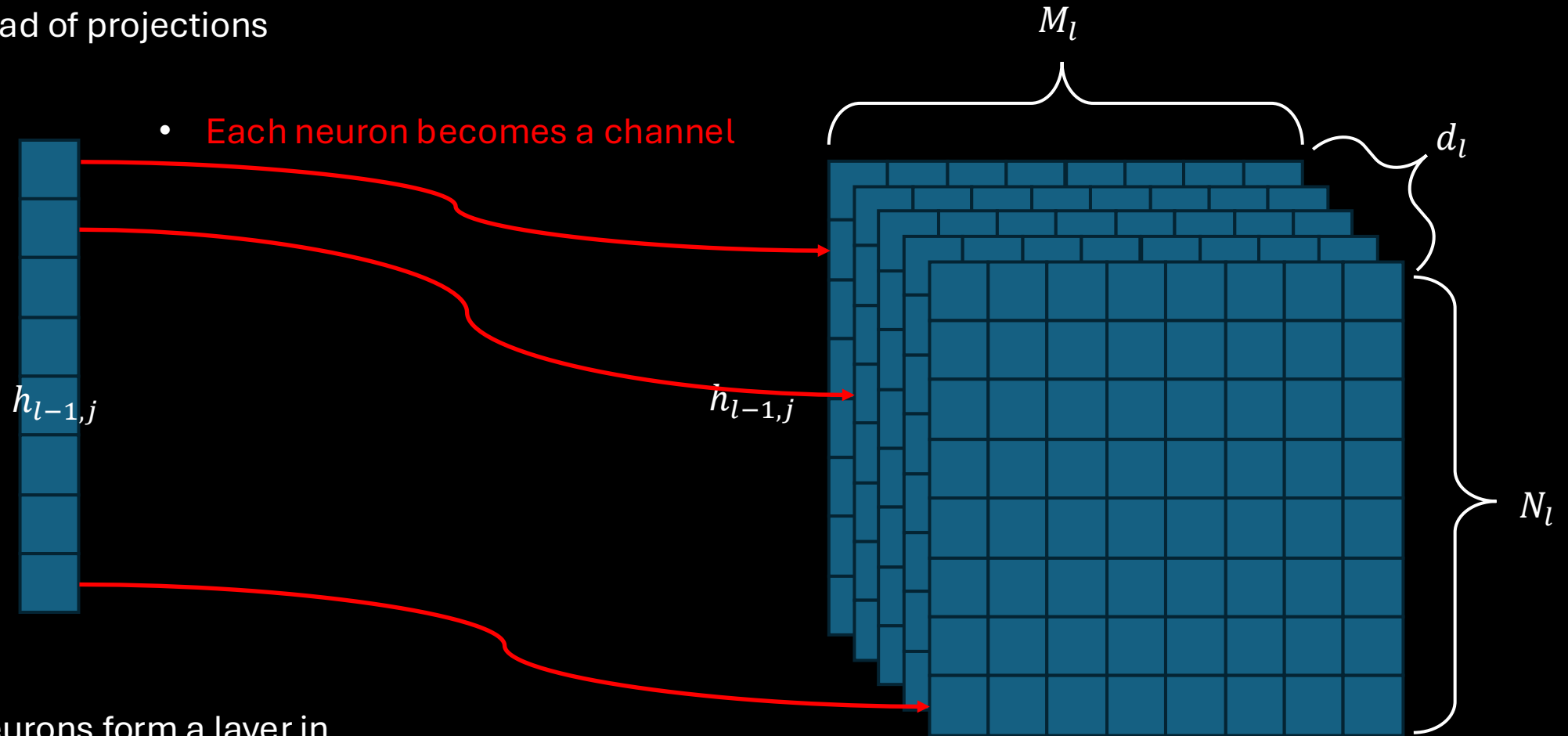
$$a_{l,k} = \sum_j w_{l,kj} * h_{l-1,j} + b_{l,k}$$

- Each $h_{l-1,j}$ is an image of neurons corresponding to the j^{th} channel at layer $l - 1$, and so is $a_{l,k}$
- There is a separate convolutional kernel linking images $h_{l-1,j}$ and $a_{l,k}$. Same kernel applies to the entire image of neurons.
- In the literature, $h_{l-1,j}$ are called channels
- $w_{l,kj}$ are convolutional filters

- Nonlinearities following activations remain similar

Convolutional layers

- Convolutions instead of projections

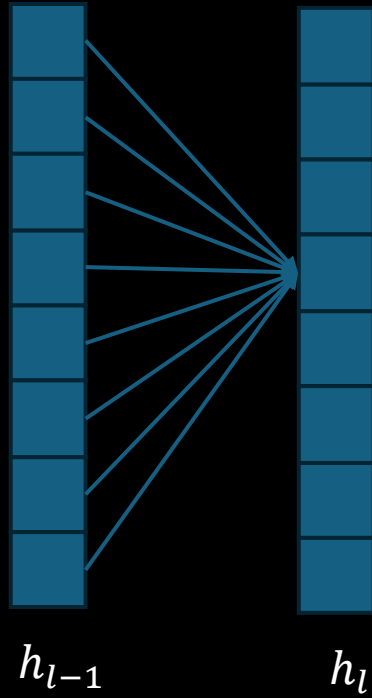


- Vector of neurons form a layer in fully connected networks

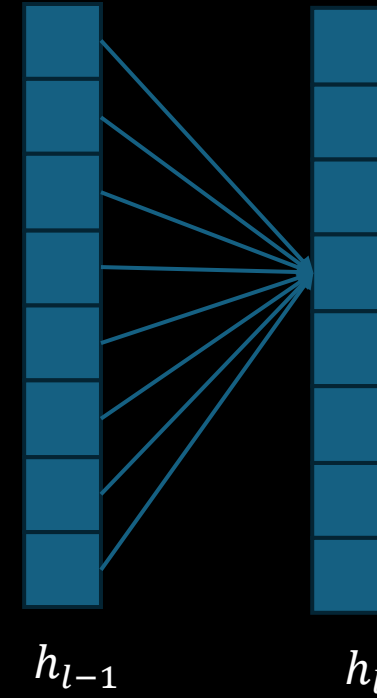
- Image of neurons form a channel in convolutional networks

Convolutional layers

- We can also use the same graphical representation



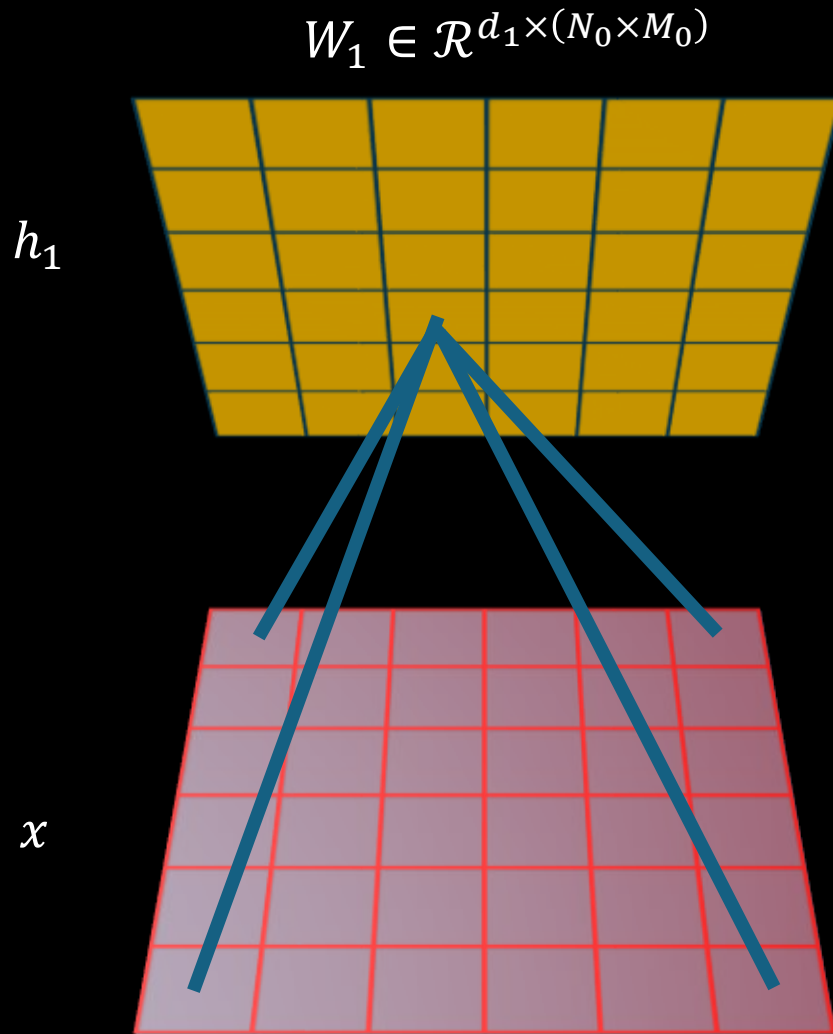
- Fully connected network



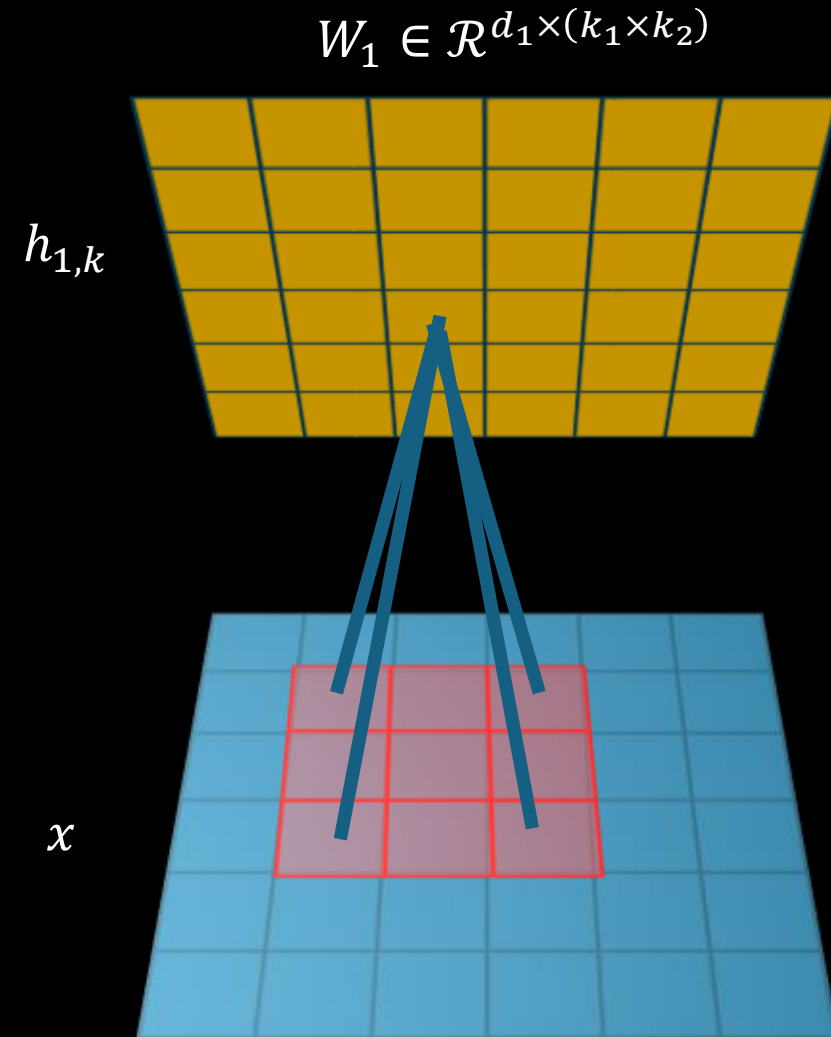
- Convolutional network

Convolutional layers

- Sparser connection



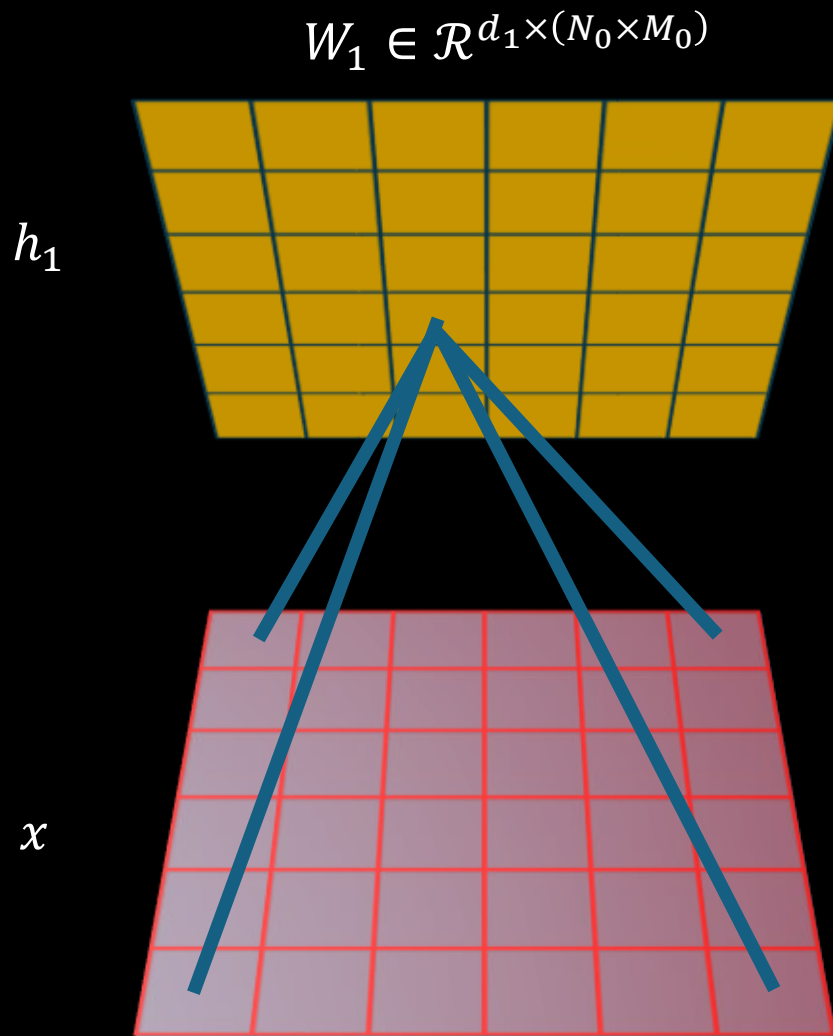
- Fully connected network



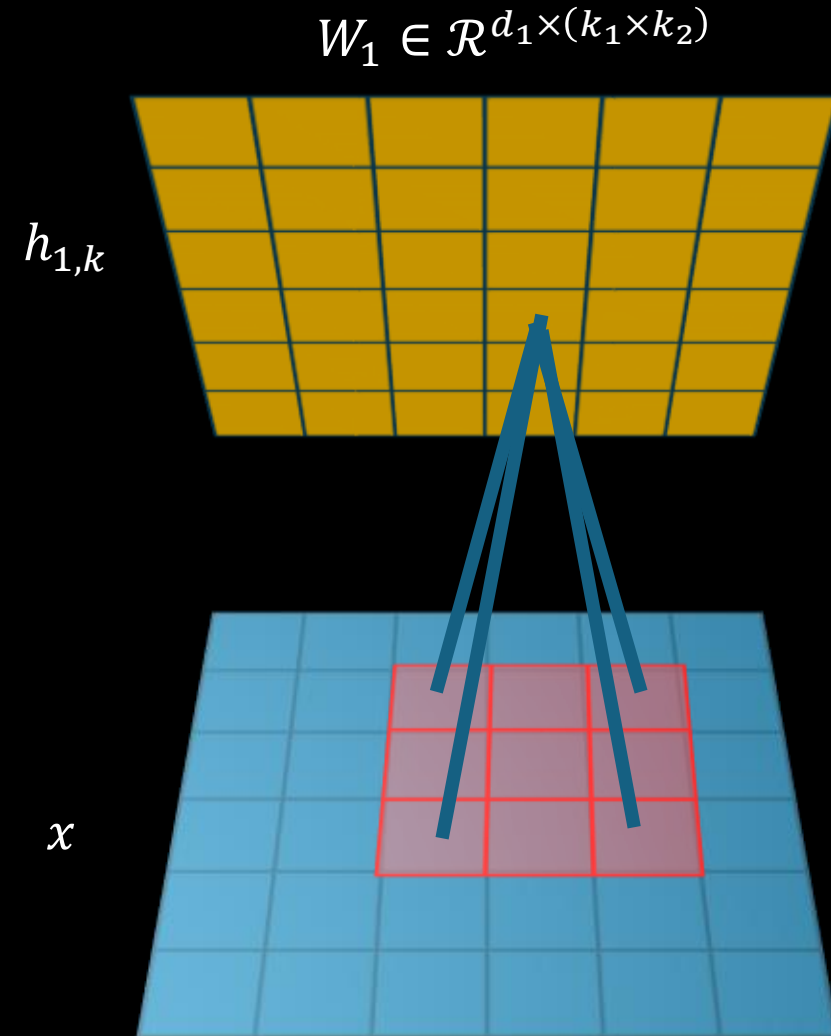
- Convolutional network

Convolutional layers

- Sparser connection



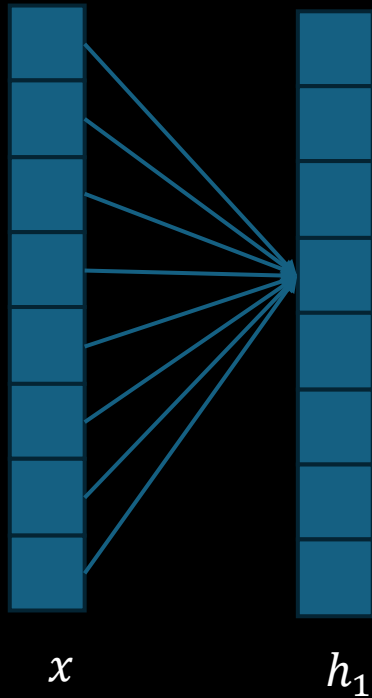
- Fully connected network



- Convolutional network

Convolutional layers

- Layer size and the number of parameters at the first layer



- Fully connected**

$$x \in \mathcal{R}^{3 \times N_0 \times M_0}$$

$$h_1 \in \mathcal{R}^{d_1}$$

$$W_1 \in \mathcal{R}^{d_1 \times (3 \times N_0 \times M_0)}$$

- Convolutional**

$$x \in \mathcal{R}^{3 \times N_0 \times M_0}$$

$$h_1 \in \mathcal{R}^{d_1 \times (N_1 \times M_1)}$$

$$W_1 = \{w_{1,1}, w_{1,2}, \dots, w_{1,d_1}\}$$

$$w_{1,j} \in \mathcal{R}^{3 \times k_1 \times k_2}$$

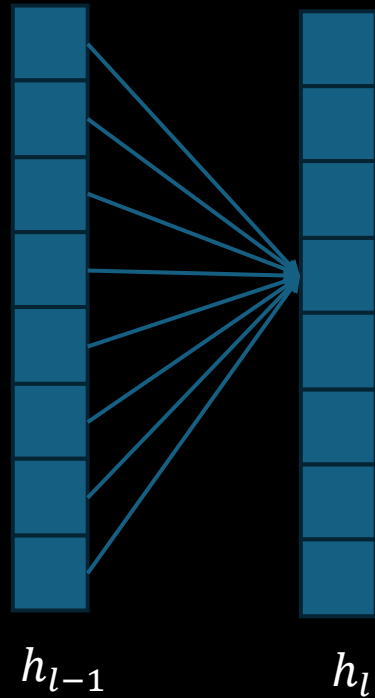
$3 \times k_1 \times k_2$: kernel
size

$$W_1 \in \mathcal{R}^{d_1 \times (3 \times k_1 \times k_2)}$$

- Larger layers with sparser connections with lower number of parameters

Convolutional layers

- Layer size and the number of parameters at the intermediate layer



- Fully connected**

$$h_{l-1} \in \mathcal{R}^{d_{l-1}}$$

$$h_l \in \mathcal{R}^{d_l}$$

$$W_l \in \mathcal{R}^{d_l \times d_{l-1}}$$

- Convolutional**

$$h_{l-1} \in \mathcal{R}^{d_{l-1} \times N_{l-1} \times M_{l-1}}$$

$$h_l \in \mathcal{R}^{d_l \times (N_l \times M_l)}$$

$$W_l = \{w_{l,jk}\}$$

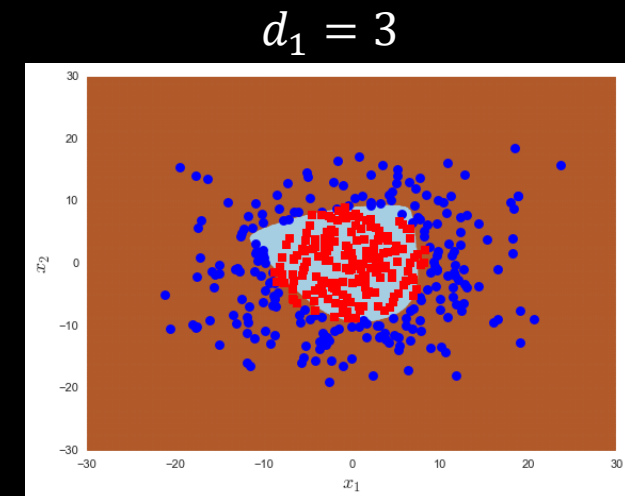
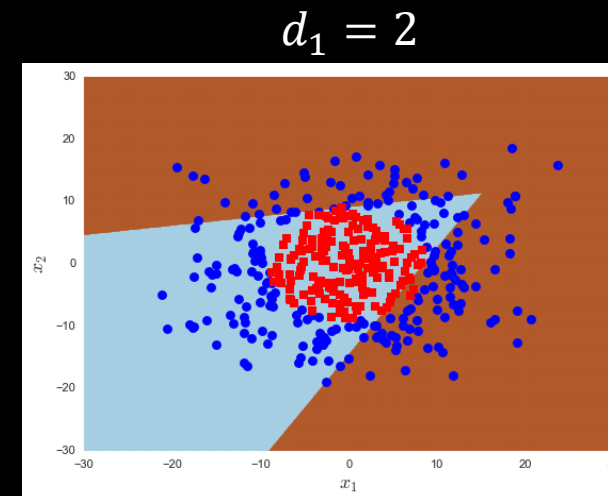
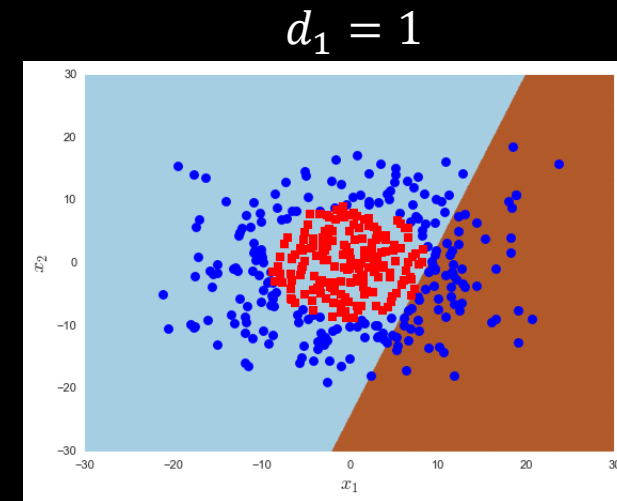
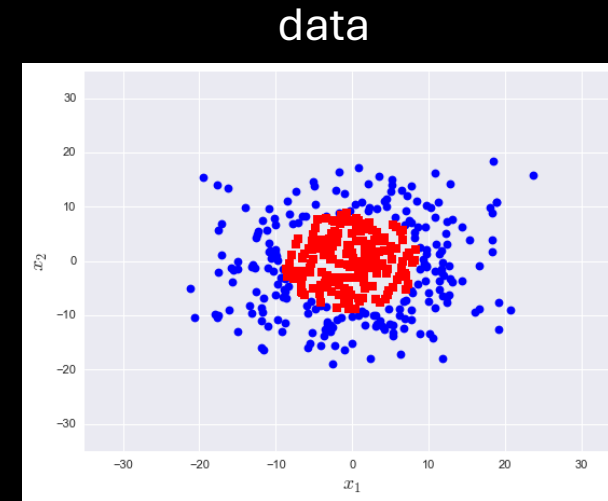
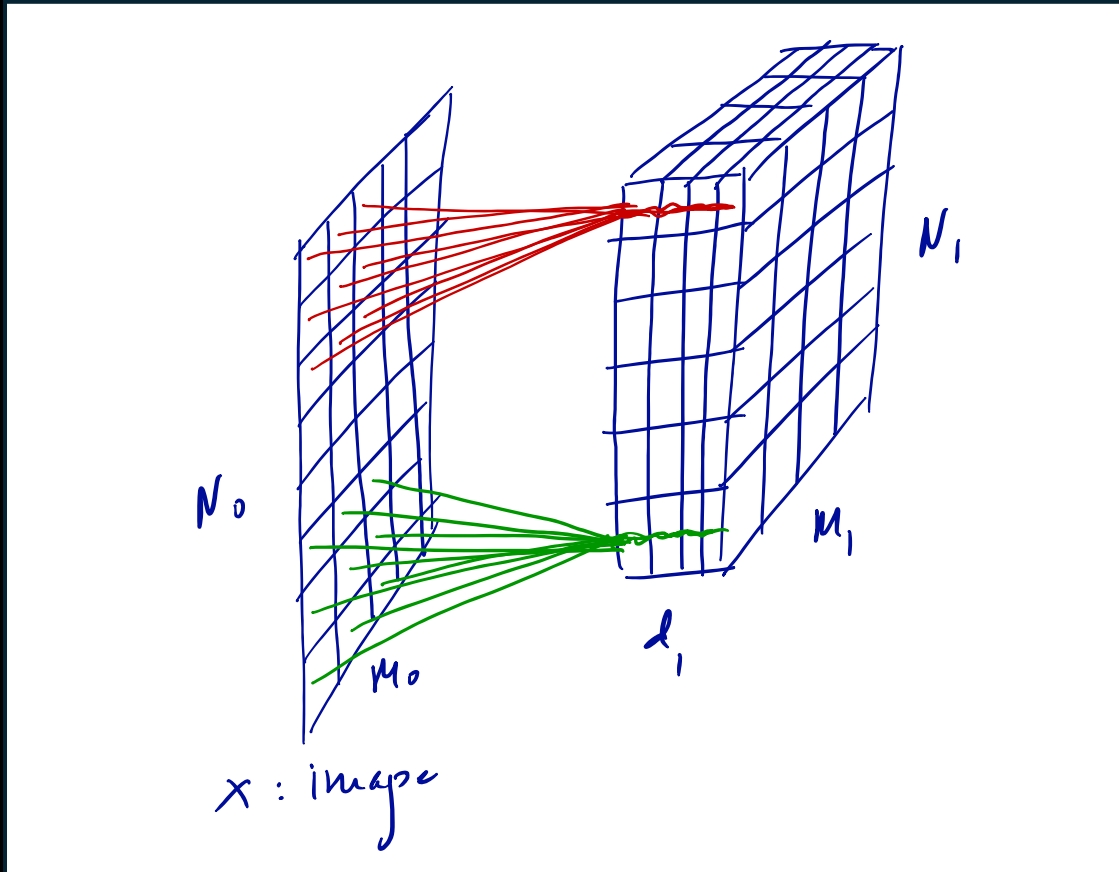
$$j = 1 \dots d_{l-1} \quad k = 1 \dots d_l$$

$$W_l \in \mathcal{R}^{d_l \times d_{l-1} \times (k_1 \times k_2)}$$

- Larger layers with sparser connections with lower number of parameters

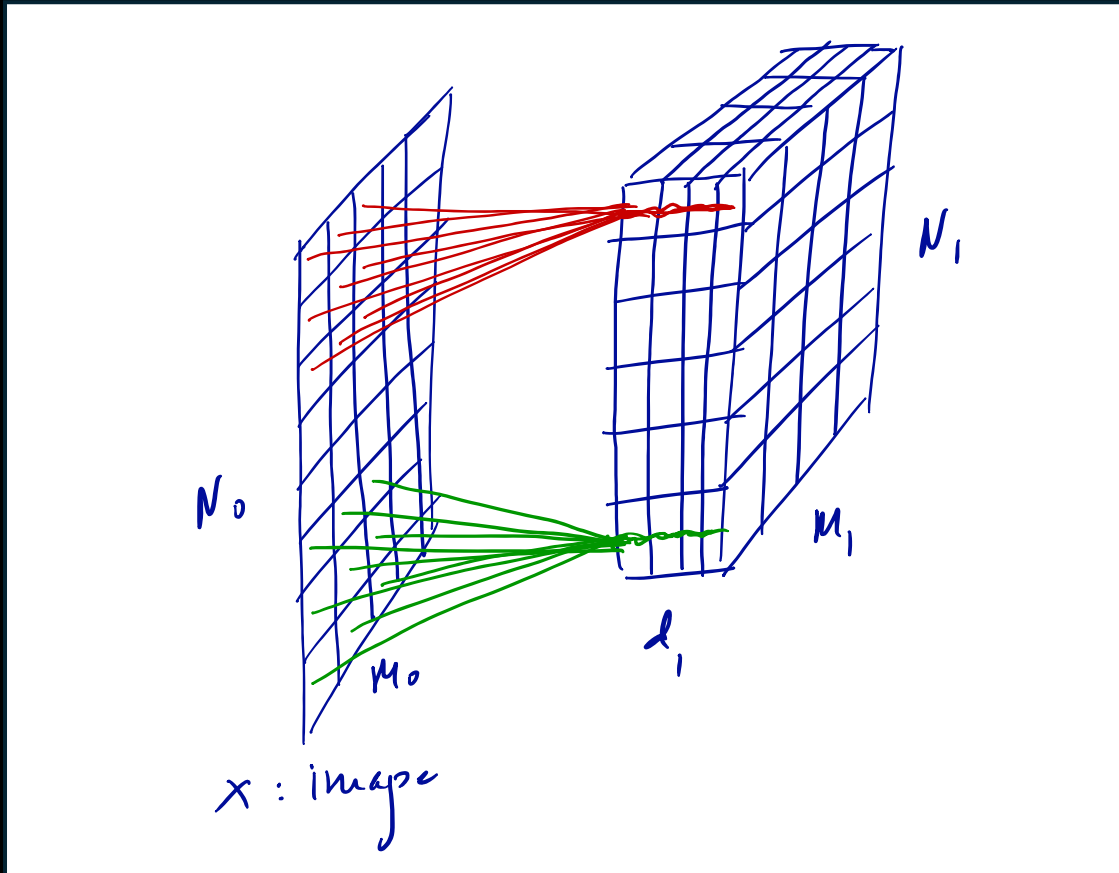
Convolutional layers

- Now we have the tool that allow us to take the images and increase dimensions without needing enormous amount of parameters.

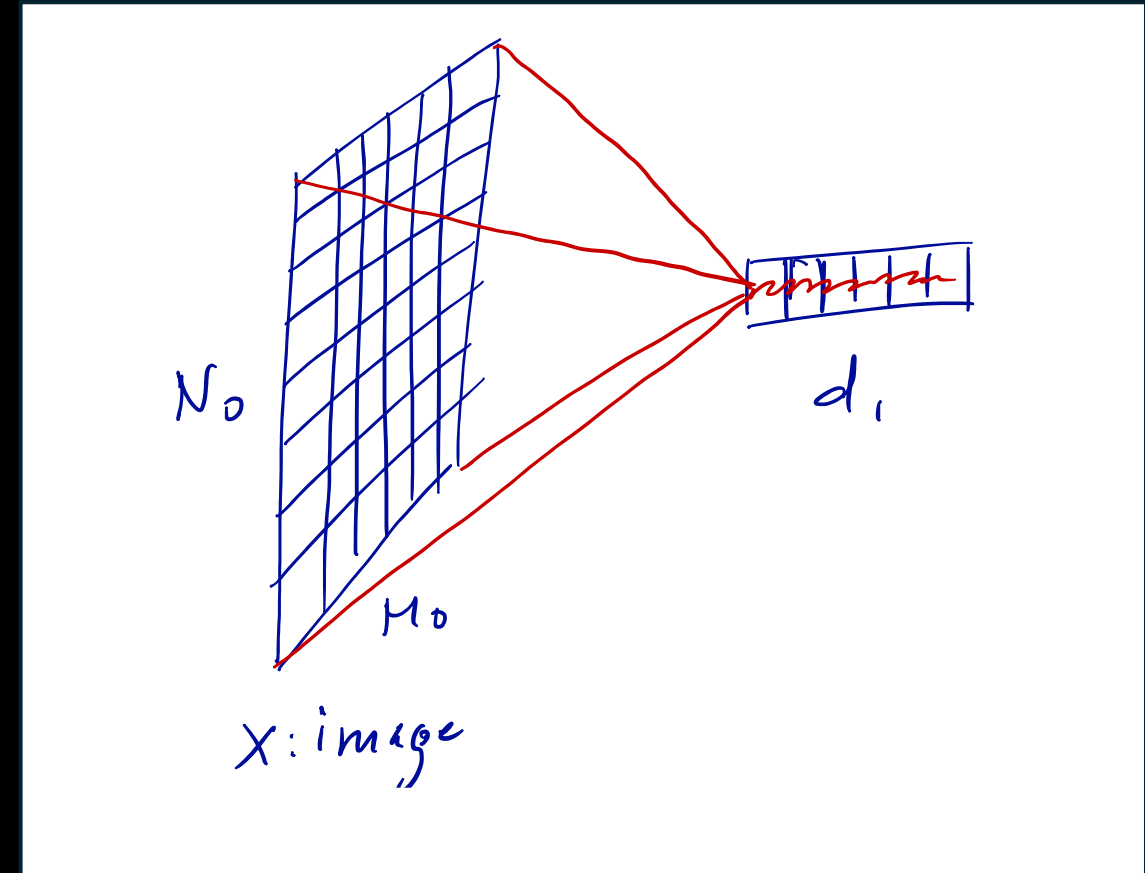


Convolutional layers

- Local vs. global information gathering



- Local neighborhoods are represented in a single vector
- Don't need large d_1 to retain information



- Entire image needs to be represented in a single vector
- Need large d_1 to retain a similar level information

Convolutional layers

- Channel size is linked with kernel size and type of convolution

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- When the kernel is placed in the image – no problem

Convolutional layers

- Channel size is linked with kernel size and type of convolution

	544	552	570	585	600	607	608	581	558	577
	549	561	595	617	610	601	595	562	545	563
	579	574	554	538	556	598	614	596	588	582
	529	514	486	476	483	509	552	584	604	586
	506	499	468	421	459	547	588	596	598	603
	567	561	519	484	510	557	586	612	603	565
	579	594	581	563	557	553	572	587	575	575
	590	601	594	586	580	563	559	587	602	585
	596	602	602	595	586	585	592	577	545	557
	593	614	589	568	588	625	610	546	519	557

- When the kernel is placed in the image – no problem
- When it is placed on the boundary – the out of boundary values are not well defined
- Two options:
 - Valid convolution: only evaluate the convolution when all the elements are defined
 - Same convolution (Padding): pad the boundaries so that results of the convolution will have the same size

Convolutional layers

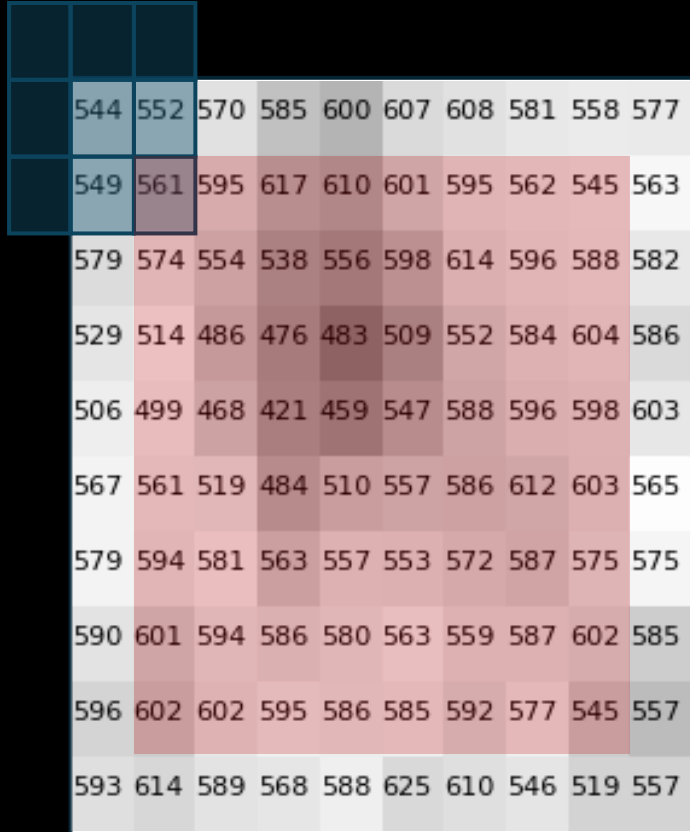
1. Valid convolution: only evaluate the convolution when all the elements are defined

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- The kernel is only evaluated within the red area

Convolutional layers

1. Valid convolution: only evaluate the convolution when all the elements are defined



- The kernel is only evaluated within the red area
- We lose a pixel at each end of the image

$$h_l \in \mathcal{R}^{d_l \times (N_l \times M_l)}$$

$$h_{l-1} \in \mathcal{R}^{d_{l-1} \times (N_{l-1} \times M_{l-1})}$$

$$w_{l,jk} \in \mathcal{R}^{d_l \times d_{l-1} \times (k_1 \times k_2)}$$

$$N_l = N_{l-1} - k_1 + 1$$

$$M_l = M_{l-1} - k_2 + 1$$

Convolutional layers

2. Same convolution (Padding): pad the boundaries so that results of the convolution will have the same size

	544	552	570	585	600	607	608	581	558	577
	549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582	
529	514	486	476	483	509	552	584	604	586	
506	499	468	421	459	547	588	596	598	603	
567	561	519	484	510	557	586	612	603	565	
579	594	581	563	557	553	572	587	575	575	
590	601	594	586	580	563	559	587	602	585	
596	602	602	595	586	585	592	577	545	557	
593	614	589	568	588	625	610	546	519	557	

- Alternatively, you can pad the image on the boundaries so that the channels will have the same size across layers

$$h_{l-1} \in \mathcal{R}^{d_{l-1} \times (N_{l-1} \times M_{l-1})}$$

$$h_{l-1} \in \mathcal{R}^{d_{l-1} \times (N_{l-1} + k_1 - 1) \times (M_{l-1} + k_2 - 1)} \quad \text{After padding}$$

$$h_l \in \mathcal{R}^{d_l \times d_{l-1} \times (N_l \times M_l)}$$

$$M_l = M_{l-1}$$

$$N_l = N_{l-1}$$

- The value you pad is a parameter. Zero is commonly used but one can also use symmetric padding

Convolutional layers

Question: Number of parameters

- Classification network with 16 possible output classes
- How many parameters are in the following fully connected networks, where each arrow is a fully connected link and assuming a bias number per layer

$$x \in \mathcal{R}^{64 \times 64} \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 16$$

- How many parameters and how many hidden neurons in each layer in the following convolutional neural network, where double arrows are “valid” convolutional links with 5x5 kernels, arrows are fully connected links, and assuming a bias number per channel for convolutional links

$$x \in \mathcal{R}^{64 \times 64} \Rightarrow 256 \Rightarrow 128 \Rightarrow 64 \rightarrow 16$$

Convolutional layers

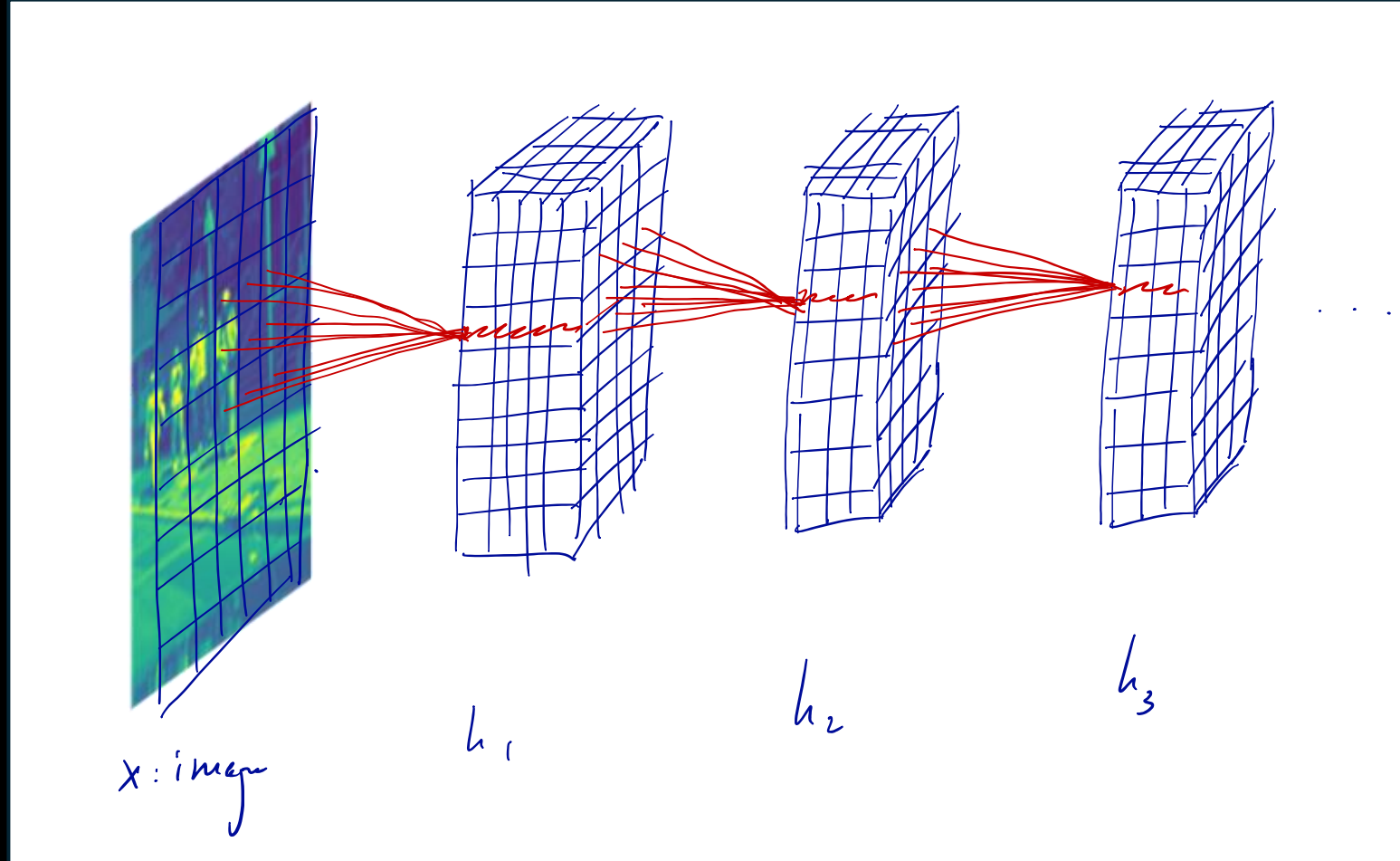
- Problems Using MLPs with grid-like structures

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

- The linear transformation has three issues
 1. ~~Fully connected links leads to too many parameters~~
 2. Images are composed of a hierarchy of local statistics
 3. Lack of translation invariance

Convolutional layers

- Hierarchically aggregating local spatial features – receptive field



Extracting task specific features from the images.
As the layers progress, more global information is encoded.

Convolutional layers

- Hierarchically aggregating local spatial features



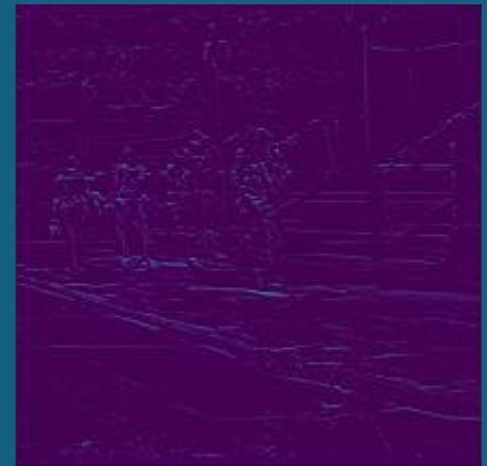
Simplified example

Layer 1



Derivatives
followed by ReLu

Layer 2



Different channels in the 2nd layer
Weighted sums of 2nd order derivatives

Convolutional layers

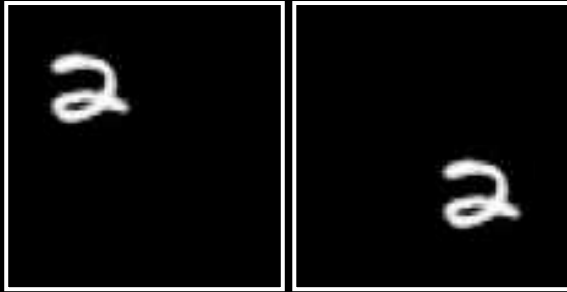
- Problems Using MLPs with grid-like structures

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

- The linear transformation has three issues
 1. ~~Fully connected links leads to too many parameters~~
 2. ~~Images are composed of a hierarchy of local statistics~~
 3. Lack of translation invariance

Convolutional layers

- Translation invariance is NOT native to fully connected networks

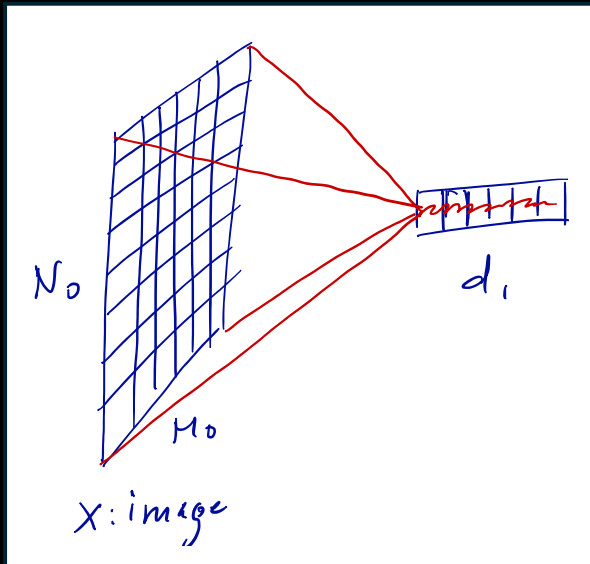


- These images will most likely lead to a very different activations in the hidden layer
- Rest of the network will see different activations
- In various vision applications, these images should lead to identical outputs

- Question: what are some applications where these two images should lead to

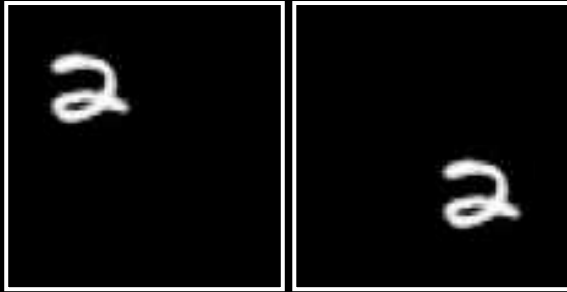
1. Identical outputs
 - Recognition
 - Detection
2. Different outputs
 - Localization
 - Segmentation

Note that we would like to have the same output but in different location for these applications

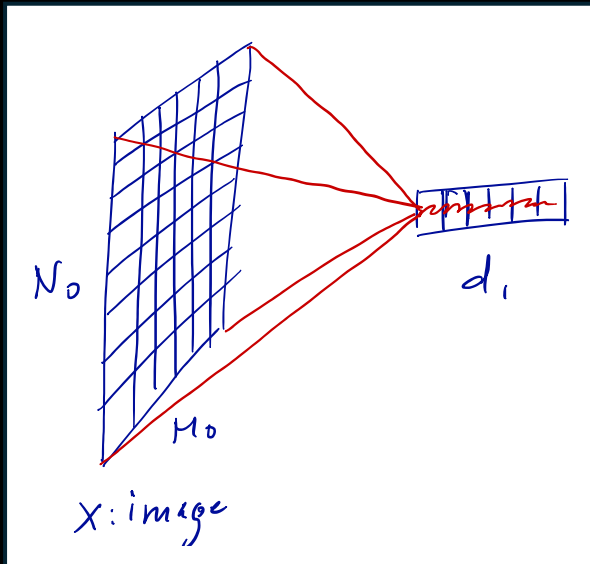


Convolutional layers

- Translation invariance is NOT native to fully connected networks



- These images will most likely lead to a very different activations in the hidden layer
- Rest of the network will see different activations
- In various vision applications, these images should lead to identical outputs
- Question: Is it possible to teach a fully connected network to be invariant to translation? How?
- By applying random translations of each image in the training set
- Not the most elegant way!
- Convolution can help - it is translation equivariant



Convolutional layers

- Convolution can help - it is translation equivariant
- Translation equivariance: applying a transformation to the input yields the same result as applying the transformation to the output

$$f(T(x)) = T(f(x))$$

- Convolution is translation equivariant



- Convolution does not have translation invariance

$$f(T(x)) \neq f(x)$$

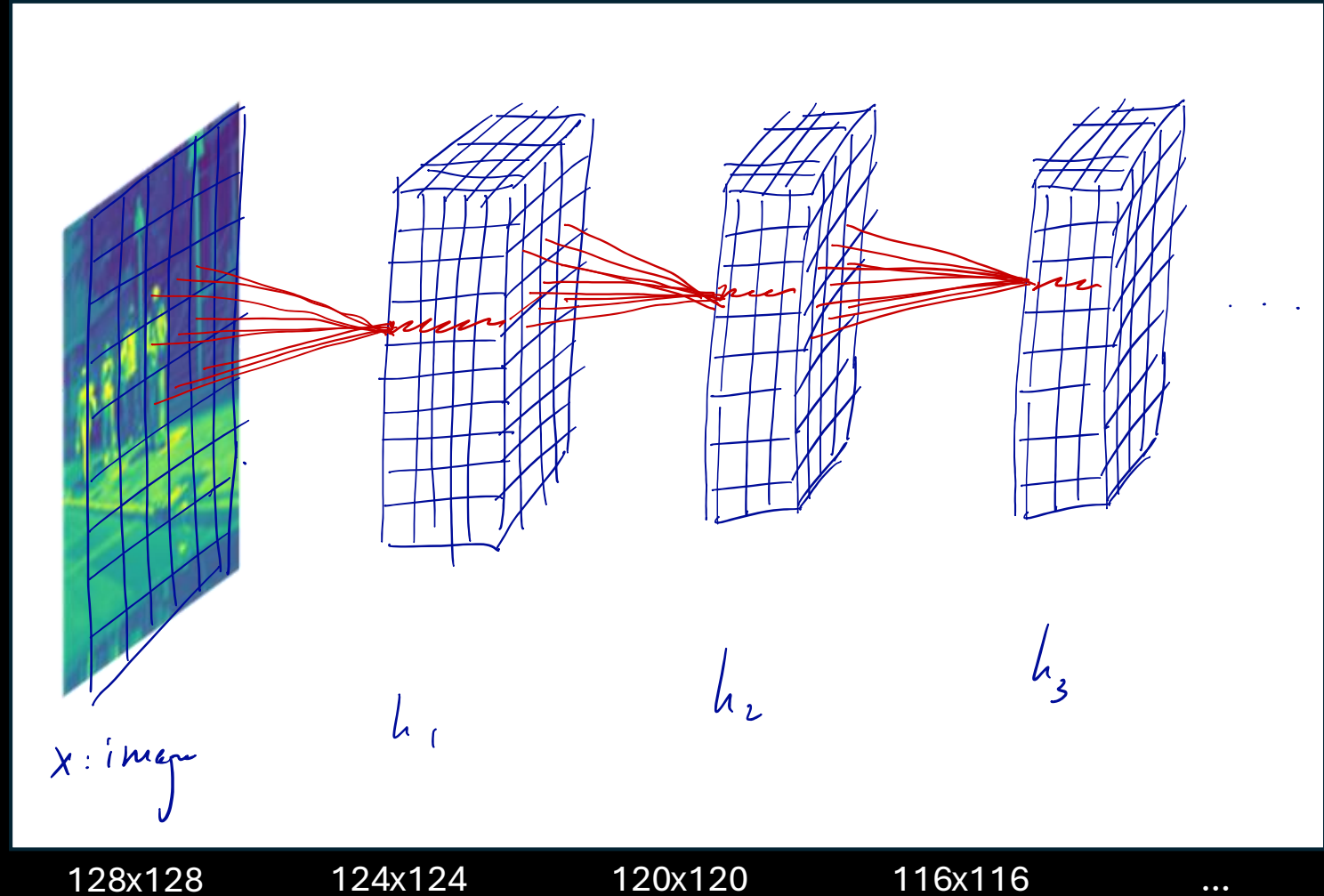
Outline of this lecture

- ✓ Motivation
- ✓ Convolutional layers
 - Strides
 - Pooling
 - A simple image classification CNN

Strides

- Convolutional layers is a great idea but

- Assume we use convolution kernels of size 5x5 and valid padding in a recognition system >> output channel size should be 1x1 (with number channels equal to the number of classes)
- You would need many layers.
- Or very large kernels.
- There might be another way to do this...



Strides

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- In a normal convolution you only move one pixel in each direction, not skipping any pixels

Strides

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- In a normal convolution you only move one pixel in each direction, not skipping any pixels

Strides

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- In a normal convolution you only move one pixel in each direction, not skipping any pixels
- However, you can decide to skip several pixels while shifting your kernel, e.g., you can skip 2 pixels

Strides

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- In a normal convolution you only move one pixel in each direction, not skipping any pixels
- However, you can decide to skip several pixels while shifting your kernel, e.g., you can skip 2 pixels
- Leads to reduction in channel size

$$N_l = \left\lfloor \frac{N_{l-1} - k_1}{s_1} \right\rfloor$$

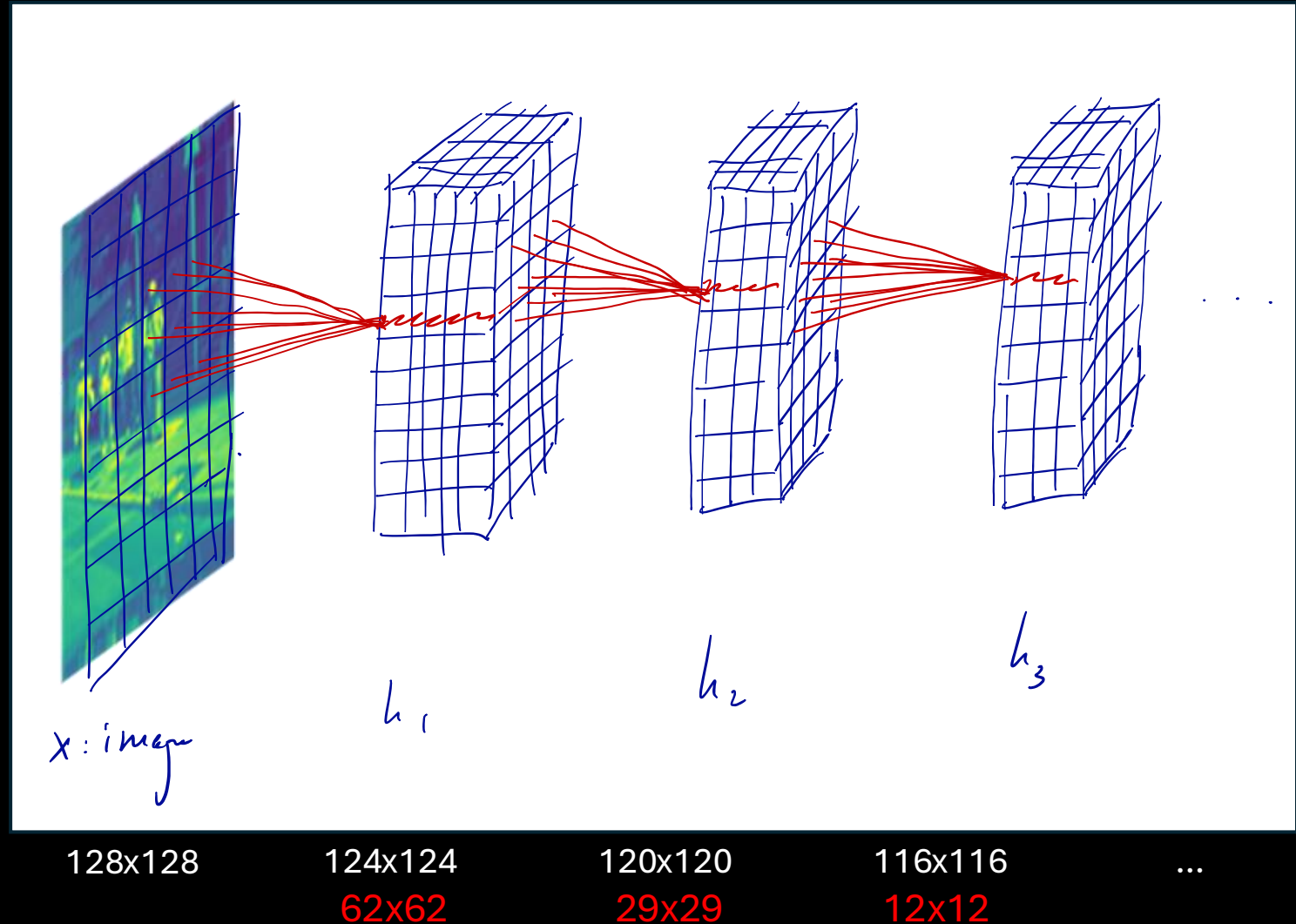
$$M_l = \left\lfloor \frac{M_{l-1} - k_2}{s_2} \right\rfloor$$

Strides

$$N_l = \left\lfloor \frac{N_{l-1} - k_1}{s_1} \right\rfloor$$

$$M_l = \left\lfloor \frac{M_{l-1} - k_2}{s_2} \right\rfloor$$

- Assume we use convolution kernels of size 5x5, valid padding **with stride 2** in a recognition system >> output channel size should be 1x1 (with number channels equal to the number of classes)
- The dimension drops very quickly. The rate of drop will be faster if stride is increased.
- You lose information. Higher stride means higher loss of information.
- You do not gain translation invariance.
- If used, it is most common to use stride 2 in all directions.



Strides

- Convolutional layers is a great idea but
- Dimensionality reduction requires many parameters
- Translation invariance would be very useful
- Strides help with dimensionality reduction but not very much with translation invariance...

Outline of this lecture

- ✓ Motivation
- ✓ Convolutional layers
- ✓ Strides
- Pooling
- A simple image classification CNN

Pooling

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- Pool information in a neighborhood
- Represents the region with one number >> summarizes information
- Applied to each channel separately
- **Max-pooling** – maximum of the activation values
- **Min-pooling** – minimum of the activation values
- Both are non-linear operations, like median filtering
- **Averaging pooling** – linear operator
- Max-pooling is the most commonly used version

Pooling

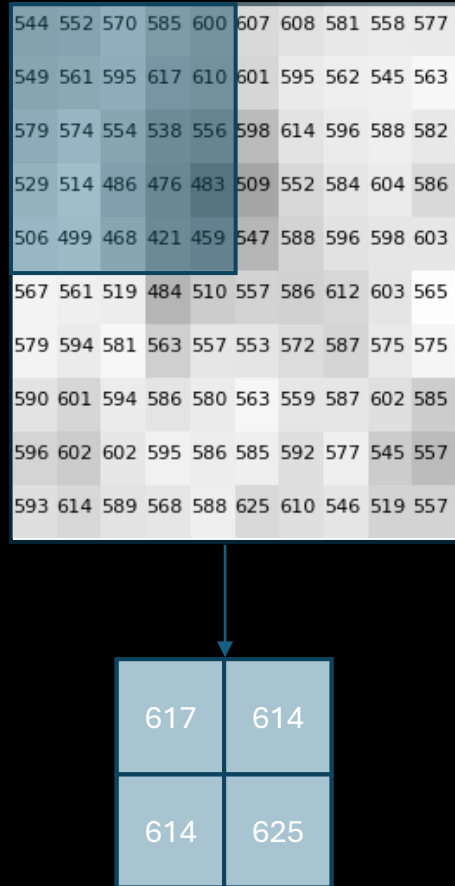
- Max pooling

544	552	570	585	600	607	608	581	558	577
549	561	595	617	610	601	595	562	545	563
579	574	554	538	556	598	614	596	588	582
529	514	486	476	483	509	552	584	604	586
506	499	468	421	459	547	588	596	598	603
567	561	519	484	510	557	586	612	603	565
579	594	581	563	557	553	572	587	575	575
590	601	594	586	580	563	559	587	602	585
596	602	602	595	586	585	592	577	545	557
593	614	589	568	588	625	610	546	519	557

- Represents the entire region with the neuron that achieves the highest activation
- Leads to partial local translation invariance
- 617 in the highlighted area can be in any of the neurons, the pooled value will not change
- Often applied with strides equal to the size of the pooling kernel . Note that the pooling kernel does not contain any learnable parameter.

Pooling

- Max pooling



- Leads to substantial dimensionality reduction
- Even when the pooling kernel is of size 2x2, it can halve the image!
- As the size of the pooling kernel increases, the reduction increases as well
- Non-linear dimensionality reduction
- Only the most prominent activation is transmitted to the next layer
- More advanced pooling mechanisms exist, e.g. CapsuleNets [Sabour, Frost and Hinton 2017]

Convolutional layers

- Problems Using MLPs with grid-like structures

$$h^l = \sigma_l(W_l h^{l-1} + b_l)$$

- The linear transformation has three issues
 1. Fully connected links leads to too many parameters
 2. Images are composed of a hierarchy of local statistics
 3. Lack of translation invariance – partially solved

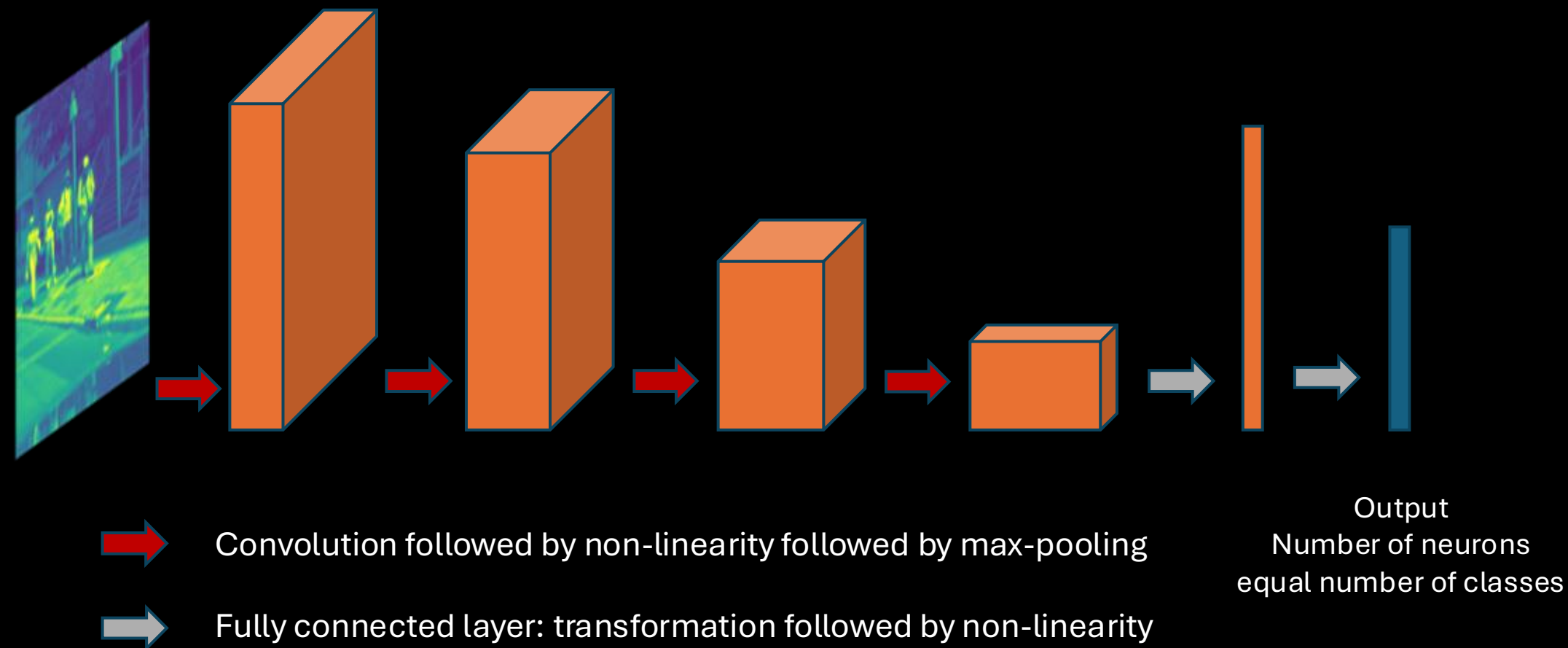
Outline of this lecture

- ✓ Motivation
- ✓ Convolutional layers
- ✓ Strides
- ✓ Pooling
- A simple image classification CNN

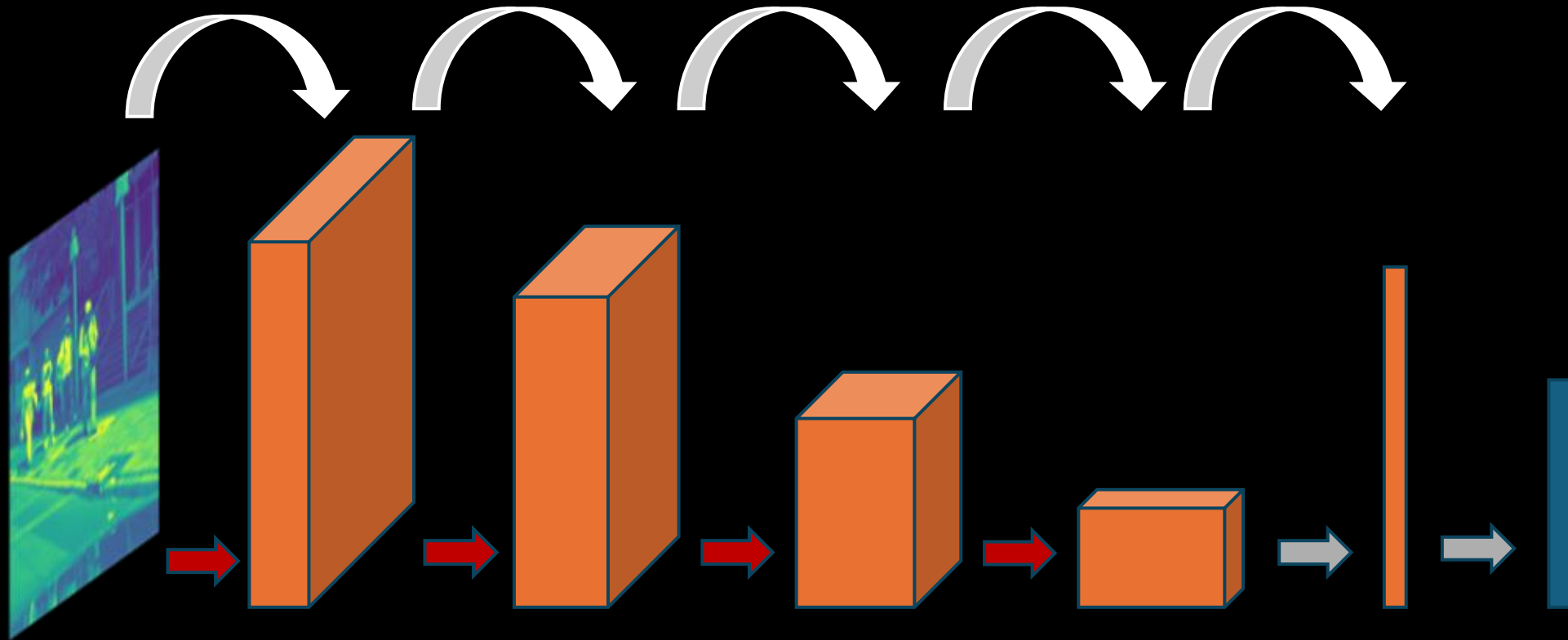
A simple image classification CNN



A simple image classification CNN

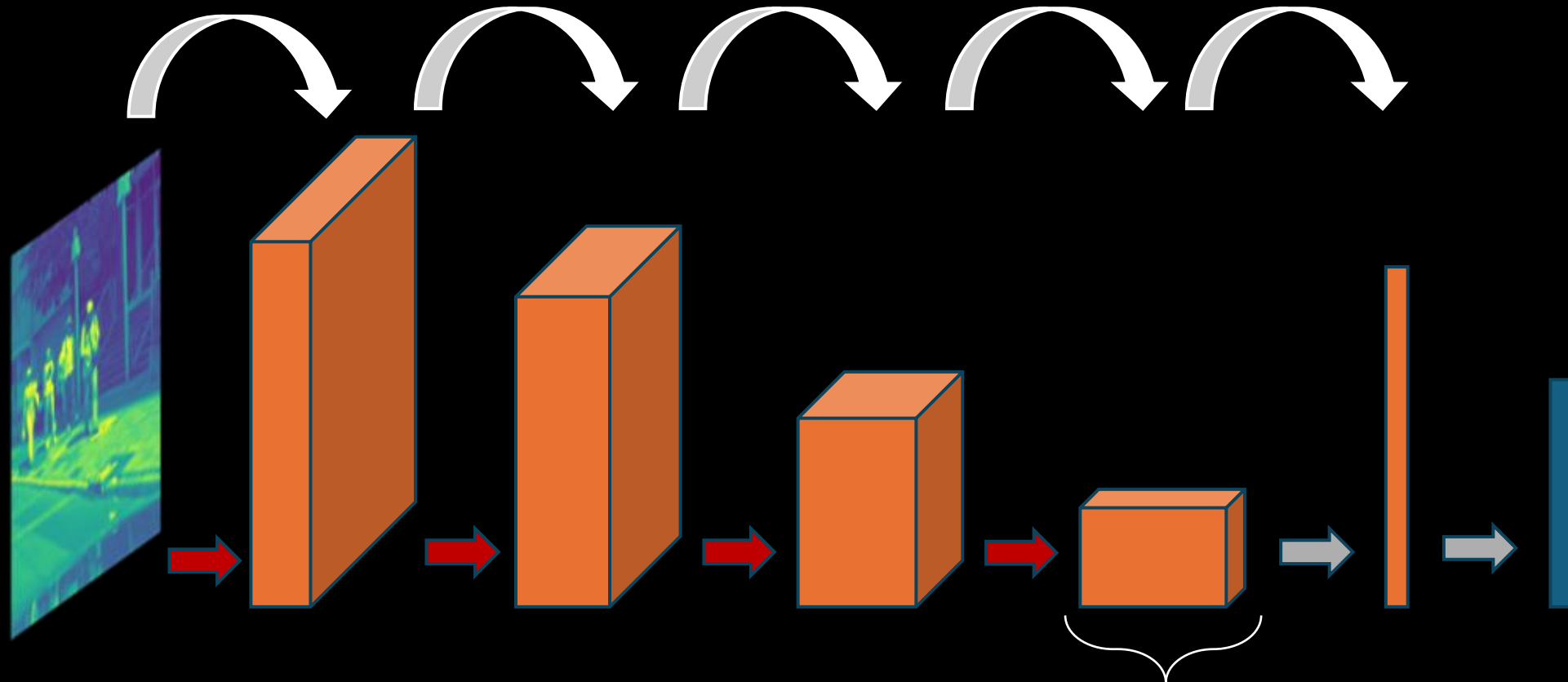


A simple image classification CNN



- Local features are extracted and aggregated throughout the network
- The last layers “sees” the entire image and the features encode global information

A simple image classification CNN



- This channel must see large areas of the image so we can determine semantic labels for each image.
- Due to the preceding convolutions and pooling each neuron in this channel sees a large area of the image – it has a large **receptive field**

Outline of this lecture

- ✓ Motivation
- ✓ Convolutional layers
- ✓ Strides
- ✓ Pooling
- ✓ A simple image classification CNN